海云智视　基于 PYNQ-Z2 的船舶智能监测系统

摘要

本作品为自主选题，面向岸港与航道船舶监测，构建由 PYNQ-Z2、以太网链路和电脑上位机组成的软硬件协同系统。实际板端部署采用 YOLOv4-Tiny：FPGA 可编程逻辑执行卷积和采样等计算，ARM 完成网络调度、解码、非极大值抑制、ByteTrack 目标关联与 TCP 通信。电脑端支持数据集独立图像检测、连续图像序列追踪、警戒区和越线告警、单目深度预测，以及摄像头在线检测与追踪。

船舶检测模型针对散货船、渔船、普通货船、矿砂船、客船和集装箱船六类目标，输入尺寸为 416×416。电脑将图片转换为 RGB、CHW 排列的 uint8 张量，以 CRC32 校验后通过以太网上传；板卡返回类别、置信度、检测框、目标 ID 和阶段耗时。连续场景在同一批次中保持 ByteTrack 状态，完成日光、多船并行、交会遮挡、低对比度、远处小目标、低能见度和夜间条件下的跨帧编号。

本项目同时开展 YOLOv4-Tiny 板端部署、YOLOv5 模型优化、ByteTrack 多目标追踪与单目深度预测。YOLOv5 部分引入 SimAM、CARAFE 和 C3_DCN，形成 YOLOv5-SSC 并进行检测实验；YOLOv4-Tiny 部分完成定点数据处理、HLS 算子映射和 ARM 调度。单目深度采用 Depth Anything V2 ViT-S，在船框内部统计深度并输出前向距离估计。各模块通过统一帧号接入上位机，完成检测、追踪、告警、深度显示和摄像头交互。

关键词：FPGA；PYNQ-Z2；YOLOv4-Tiny；ByteTrack；船舶检测；单目深度；以太网。

第一部分　作品概述

本项目围绕固定岸侧视角下的船舶识别与连续监测开展自主设计。人工查看航道画面难以持续记录每艘船的位置与运动过程，水面反光、远处小目标、遮挡及低照度又会影响视觉识别。我们将船舶检测、跨帧关联和可交互事件告警组合

为一套可在 FPGA 板卡上运行的监测系统，并通过以太网接入电脑图片与摄像头。

YOLOv4-Tiny 采用单阶段目标检测方法，通过一次网络计算同时预测船舶的位置、类别和置信度，再经过后处理得到最终检测结果。其轻量化结构适合资源受限的硬件平台[1]。ByteTrack 通过高低分检测框关联保持连续目标身份[2]；Depth Anything V2 提供单图深度表征[3]。本项目以这些公开方法为基础，重点完成船舶任务适配、HLS 算子实现、ARM 追踪与网络传输，以及上位机交互集成。本项目完成了 基于 YOLOv5[4]的 YOLOv5-SSC 船舶检测优化实验与 YOLOv4-Tiny 板端部署，并实现目标追踪、区域预警、单目深度预测及摄像头实时交互。

1.1 功能与特性

系统围绕船舶监测中的目标识别、连续追踪、事件告警与空间信息展示，形成五项相互衔接的功能。首先，对输入的独立图像执行船舶检测，输出目标位置、类别及置信度，为后续分析提供基础。其次，对连续图像序列进行检测与跨帧关联，为船舶分配目标编号并绘制运动轨迹，便于观察多船并行、交会遮挡等情况下的目标变化。在此基础上，用户可通过上位机设置矩形警戒区或越线规则；当船舶满足触发条件时，系统生成告警记录，关联目标编号与触发帧，并支持点击记录回看对应画面。

此外，系统提供单目深度预测功能，由电脑端根据输入图像生成深度图，并结合船舶检测框给出目标前向距离估计，辅助观察场景中的远近关系。摄像头接入功能则将输入扩展至现场采集画面，通过以太网将图像发送至板卡处理，持续更新检测与追踪结果。上位机统一组织连接管理、数据选择、模式切换、运行控制、结果回放和视频导出。（见图 1-1）

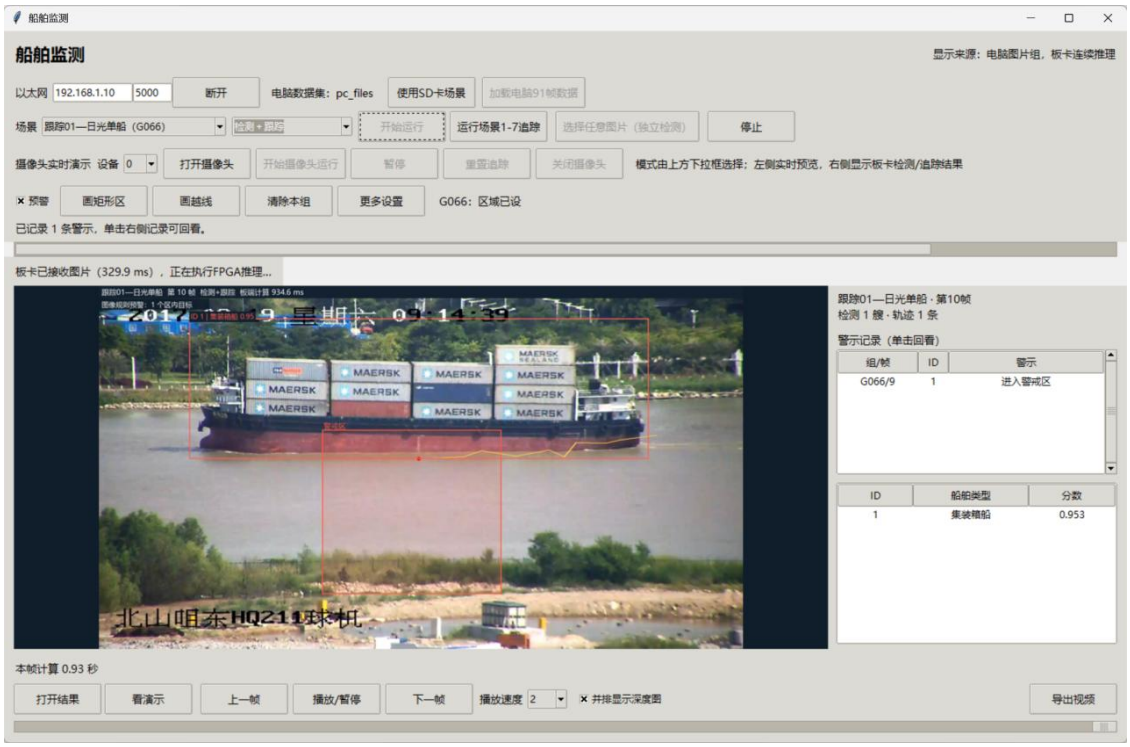


图 1-1　船舶监测上位机功能界面

1.2 应用领域

本系统面向岸港、内河航道、码头出入口及船闸等固定视角下的船舶监测需求，为船舶识别、轨迹观察和区域事件记录提供支持。在岸侧与航道监测中，系统通过检测获取船舶类别和位置，利用连续追踪关联同一目标在不同帧中的运动状态，帮助监控人员观察多船并行、交会及遮挡过程。在码头出入口和船闸周边，可根据管理需求设置警戒区或越线规则，对船舶进入指定区域、越过设定边界等事件进行提示，记录目标编号与触发画面，便于核查。

系统采用板端计算与电脑交互相结合的方式，支持图像资料的批量处理和摄像头画面的在线输入，可用于资料分析与现场功能验证。单目深度预测提供场景深度图及船舶前向距离估计，为观察目标远近关系补充信息。上位机将船舶类别、位置、轨迹和告警记录集中展示，并提供结果回放与视频导出功能，方便监控人员查看船舶动态、核对事件过程和整理监测资料。系统将图像采

集、目标分析、事件提示与结果追溯衔接起来，为航道日常巡查及港口区域的视觉监测提供技术支持。（见图 1-2）



图 1-2　固定岸侧视角下的船舶监测应用示意

1.3 主要技术特点

本系统采用 PL、PS 与电脑端协同架构，将船舶检测、连续追踪、区域告警和单目深度预测相结合，通过硬件加速与软件调度完成图像处理和结果交互。

在模型选型方面，统一采用 416×416 输入的 SeaShips[5]测试结果表明，YOLOv4-Tiny 的 mAP50、召回率和 F1 分别为 93.95%、90.26%和 91.01%，较改进 YOLOv5 提高 11.60、14.76 和 10.09 个百分点，漏检数量减少 60.24%。结合目标检出对追踪与告警的影响，选择本次测试中综合表现更好的 YOLOv4-Tiny 进行板端部署。

在实现方面，PL 通过 HLS 实现卷积、池化和上采样等算子，并在网络层间复用加速模块；PS 负责推理调度、检测后处理和 ByteTrack 追踪，通过高、低置信度检测框的分阶段关联维护目标身份。以太网连续批次传输支持逐帧处

理，并在批次内保持追踪状态。电脑端负责图像输入、结果展示、告警和深度预测，通过场景标识与帧号关联各类结果，形成完整的监测流程（见表 1-1）。

1.4 主要性能指标

表 1-1 主要性能指标

评价维度	核心指标	本项目结果	说明
检测精度	mAP50	93.95%	六类船舶检测精度
目标检出能力	召回率 Recall	90.26%	衡量目标检出与漏检情况
身份保持能力	IDF1	86.16%	衡量船舶身份保持表现
跟踪稳定性	ID 切换次数 IDSW	2 次	17 组 189 帧累计切换次数
板端处理能力	FPGA 推理平均耗时	约 1.06 秒/帧	16 帧板端运行记录
硬件实现	加速器时钟	100 MHz	R3 候选通过布局布线及时序检查

海云智视实现了船舶检测、连续跟踪、串口交互与区域预警的完整流程。在现有 SeaShips 数据评价中，六类船舶检测 mAP50 达到 93.95%，召回率为 90.26%，精确率为 91.77%，F1 为 91.01%。在 17 组、189 帧人工标注序列上，优化跟踪器的 IDF1 为 86.16%，累计发生 2 次 ID 切换，体现出较好的身份保持能力。板端抽样测试中，FPGA 推理平均约 1.06 秒/帧，ARM 端解码与 NMS 约 4.75 毫秒、跟踪约 0.22 毫秒，后处理开销较小。

1.5 主要创新点

（1）算子复用与协同加速：PL 侧实现卷积等算子的跨层复用，PS 侧完成调度、后处理与追踪，适配板卡资源约束。

（2）连续图像传输与追踪：通过以太网统一接入图片和摄像头，以连续批次保持追踪状态，实现输入与结果的逐帧对应。

（3）监测与告警一体化：关联检测框、目标身份和运动轨迹，支持自定义警戒规则、事件记录与触发画面回看。

（4）算法优化与部署选型：开展 YOLOv5-SSC 优化及模型对比，结合检测表现选择 YOLOv4-Tiny 部署。 其中，YOLOv5-SSC 引入 SimAM 注意力机制[6]、CARAFE 上采样[7]和可变形卷积[8]。

（5）单目深度辅助感知：结合检测框与深度预测，补充船舶远近信息

1.6 设计流程

系统设计首先分析船舶检测、连续追踪与区域告警需求，确定总体架构及软硬件分工。随后整理船舶数据集，开展 YOLOv5 优化与模型对比，选择 YOLOv4-Tiny 进行板端部署。利用 HLS 实现卷积、池化和上采样等算子，在 Vivado 中完成硬件集成、综合与实现；通过 Vitis 开发 ARM 端调度、检测后处理、ByteTrack 及以太网通信程序。电脑端开发上位机，集成图像输入、结果显示、交互告警和单目深度预测。最后通过独立图像、连续场景和摄像头输入开展联调，验证各项功能并记录性能与资源占用（见图 1-3）。



图 1-3 系统设计与 FPGA 部署流程

第二部分 系统组成及功能说明

2.1 整体介绍

海云智视是一套基于 PYNQ-Z2 开发板的船舶检测、跟踪与预警系统，整体由电脑上位机、千兆以太网通信链路和 Zynq-7020 板端处理平台组成，如图 2-1 所示。系统采用 PC、ARM 与 FPGA 协同工作的架构，分别承担人机交互与应用分析、任务控制与算法后处理、神经网络计算加速等任务。

电脑上位机负责图片及摄像头图像输入，将图像整理为 RGB 通道、CHW 排列的 uint8 数据，通过以太网发送至板卡，并结合 CRC32 校验检查数据完整性。

板卡处理完成后，将检测框、船舶类别、轨迹 ID 及耗时等信息返回上位机，用于结果叠加显示、轨迹回放和运行状态监测。上位机同时承担 TCP 数据管理、图像空间告警和单目深度估计，其中图像空间告警依据用户设置的警戒区与越线规则触发，单目深度估计作为辅助场景感知功能。

板端以 Zynq-7020 为核心，由 PS 端 ARM Cortex-A9 处理系统与 PL 端 FPGA 加速器协同完成计算。PS 端负责输入管理与量化、推理任务调度、YOLO 输出解码、非极大值抑制（NMS）、ByteTrack 目标跟踪及网络通信，并通过 SD 卡完成系统启动与文件管理。PL 端负责卷积、逐点卷积、最大池化和上采样等运算，采用定点计算、并行处理和流水线结构实现加速，通过 AXI 接口接受控制并访问 DDR 中的计算数据。

系统运行时，上位机发送图像，PS 端接收并组织数据，调度 PL 端完成网络加速计算，再对输出进行检测后处理和目标关联，最后将结果回传电脑，实现“图像输入—硬件加速检测—连续目标跟踪—结果显示与预警”的完整处理流程。单目深度估计在 PC 端执行，其输出用于辅助分析，实际测距精度仍需结合标定及参考数据验证。

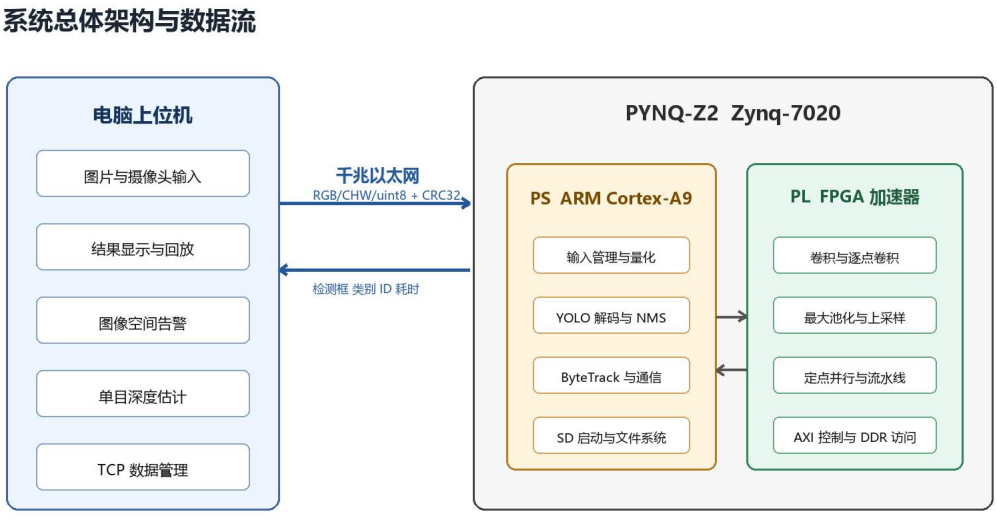


图 2-1　系统总体架构及模块关系

在上述系统架构下，各项功能由板端计算模块与电脑上位机协同完成。船舶检测与连续跟踪主要由 PL 和 ARM 共同承担，上位机负责图像采集、结果展

示、自定义告警及单目深度估计。各项功能的主要执行位置与输出内容如表 2-1 所示。

表 2-1　功能、执行位置与输出

功能	主要执行位置	输出
独立图片检测	PL + ARM	船舶类别、置信度、检测框
连续场景追踪	PL + ARM	目标 ID、当前框、历史轨迹
自定义告警	电脑	入区/越线事件、目标 ID、触发帧
单目深度	电脑 GPU/CPU	稠密深度图、船框前向距离估计
摄像头实时演示	电脑采集 + 板卡推理	实时预览、板卡检测或追踪结果

2.2 硬件系统介绍

2.2.1 硬件整体介绍

本系统以 PYNQ-Z2 开发板为计算核心，结合 SD 卡、电脑、摄像头与以太网链路，构建船舶图像输入、板端推理和监测结果展示平台。板卡采用 Zynq-7000 系列 XC7Z020 器件，将 ARM 处理系统（PS）与可编程逻辑（PL）集成于同一芯片，通过片上 AXI 互连实现控制与数据交换。板卡电路设计参见文献[9]，Zynq 处理系统及接口结构参见文献[10]。板卡外观及板载接口如图 2-2 所示。

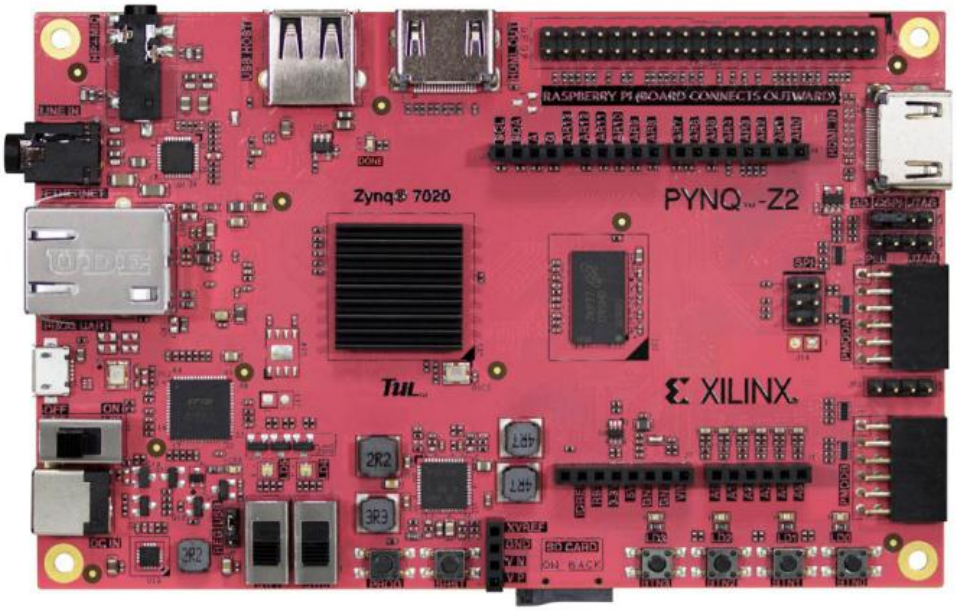


图 2-2　PYNQ-Z2 开发板全图

系统按照任务特点划分计算职责：PL 执行 YOLOv4-Tiny 网络中的卷积、池化和上采样；PS 完成逐层调度、检测框解码、非极大值抑制、ByteTrack 追踪与以太网通信；电脑端负责图片或摄像头输入、结果可视化、区域告警和单目深度预测。摄像头接入电脑，图像经预处理后通过以太网发送到板卡，处理结果再返回电脑。SD 卡保存启动文件、模型权重及配套图像，并提供另一条图像输入路径（见图 2-3）（见表 2-2）。

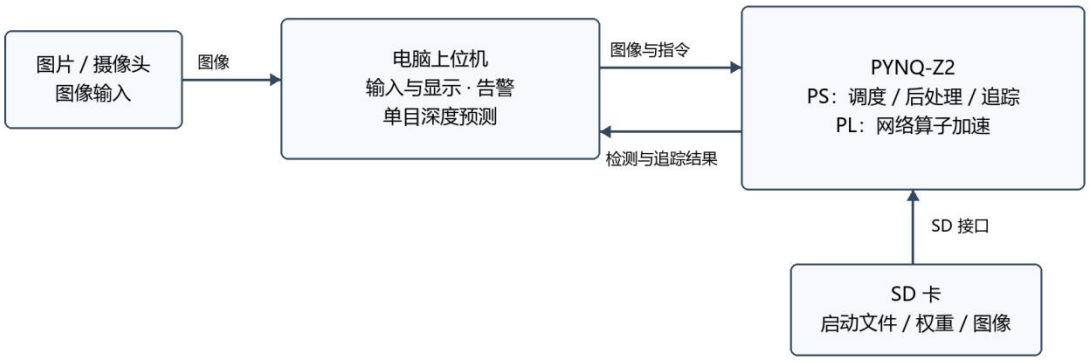


图 2-3　系统硬件组成与数据流

表 2-2　硬件组成及用途

组成	关键接口	在本系统中的用途
PYNQ-Z2	PS/PL 与 AXI	网络计算、检测后处理和目标追踪
板载 DDR3	PS DDR 接口	输入、权重、中间特征及结果缓冲
SD 卡	SD 接口	启动镜像、权重及测试输入存储
以太网链路	RJ45 / TCP	电脑与板卡双向传输
电脑与摄像头	视频采集接口	图像采集、显示、告警与深度预测

2.2.2 电路各模块介绍

（1）主控与 PS/PL 互连模块

硬件设计以 processing_system7_0 为主控模块，以 conv_0 为 HLS 计算加速模块，通过 AXI Interconnect 连接控制接口与数据接口。图 2-4 由交付工程的 Block Design 直接导出，展示加速器、PS、四组存储互连及复位模块的实际连接关系。

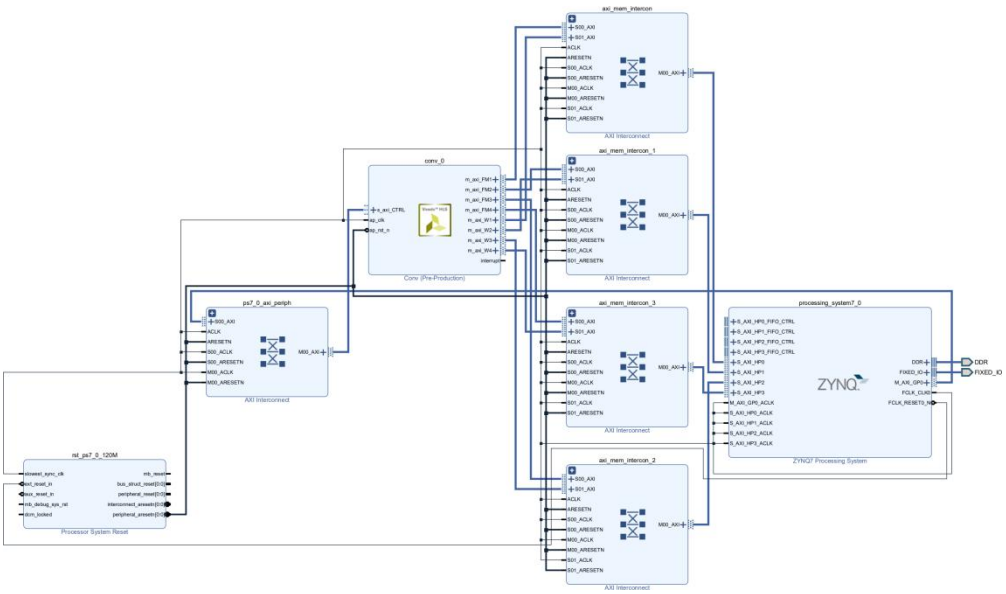


图 2-4　本项目 Vivado 硬件整体连接图

控制通路由 PS 的 M_AXI_GP0 经 ps7_0_axi_periph 连接 conv_0 的 s_axi_CTRL。ARM 通过映射寄存器写入层参数及数据地址，启动计算并读取完成状态。数据通路则由加速器的 AXI 主接口发起存储访问，经四组互连分别进入 PS 的 S_AXI_HP0～S_AXI_HP3，最终访问 DDR。控制与数据通路分开组织，

便于逐层调度和多通道访存。 PS 与 PL 的 AXI 接口及存储访问机制参见文献[10]（见表 2-3）。

表 2-3 PS/PL 关键接口

接口或信号	连接关系	作用
M_AXI_GP0	PS → 控制互连 → s_axi_CTRL	寄存器配置与状态读取
S_AXI_HP0 ～ HP3	存储互连 → PS	PL 访问 DDR 的高性能入口
FCLK_CLK0	PS → 加速 IP 及 AXI 互连	提供公共工作时钟
FCLK_RESET0_N	PS → 复位模块	生成外设与互连复位

（2）PL 计算加速模块

conv_0 由 HLS 顶层函数[11]conv 封装而成。ARM 设置 ch_in、ch_out、fsize、stride、kernel 及 act 等参数，选择当前层的通道数、特征图尺寸、步长、计算类型和激活方式。源码中 kernel=3 调用 3×3 卷积，kernel=1 调用 1×1 卷积，kernel=2 执行最大池化，上采样由其余约定分支执行。不同网络层依次复用同一加速 IP，通过更新参数与缓冲区地址完成网络前向计算（见图 2-5）。

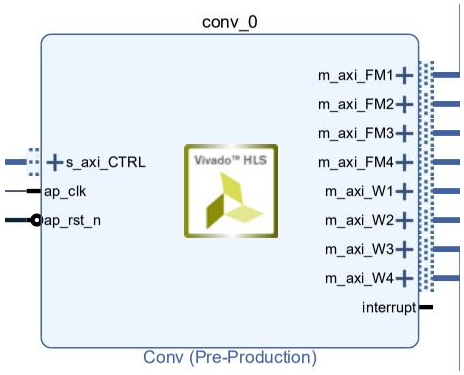


图 2-5 conv_0 接口局部放大图

控制接口采用 AXI-Lite，地址空间为 0x43C00000～0x43C0FFFF。特征图接口 m_axi_FM1～FM4 承载输入读取及输出写回，权重接口 m_axi_W1～W4 用于读取权重，其中偏置与 W1 共用接口。ap_clk 和低有效 ap_rst_n 分别用于时钟与复位。

（3）DDR 存储与数据交换模块

板载 DDR 作为 PS 与 PL 共享的数据存储空间，保存输入图像、模型权重和各层中间结果。每一组特征图与权重接口经对应的 AXI 互连汇入一条 HP 通路。ARM 负责组织存储地址与计算次序，PL 完成数据读取、运算和写回。该设计采用 HLS 生成的 AXI 主接口直接访存（见表 2-4）。

表 2-4　加速器数据通路对应关系

加速器接口	AXI 互连实例	PS 入口
FM1、W1	axi_mem_intercon	S_AXI_HP0
FM2、W2	axi_mem_intercon_1	S_AXI_HP1
FM3、W3	axi_mem_intercon_2	S_AXI_HP2
FM4、W4	axi_mem_intercon_3	S_AXI_HP3

（4）以太网通信模块

电脑与板卡之间通过 PS 侧 GEM0 以太网控制器通信。控制器经 MIO 引脚连接板载 PHY，再通过网口连接电脑。当前 PS 配置中，RGMII 信号使用 MIO16～27，MDIO 管理接口使用 MIO52～53，PHY 复位使用 MIO9。图 2-6 为 TUL 原理图[9]的 Ethernet 模块，红框标出了 PS 与 PHY 之间的主要信号（见表 2-5）。

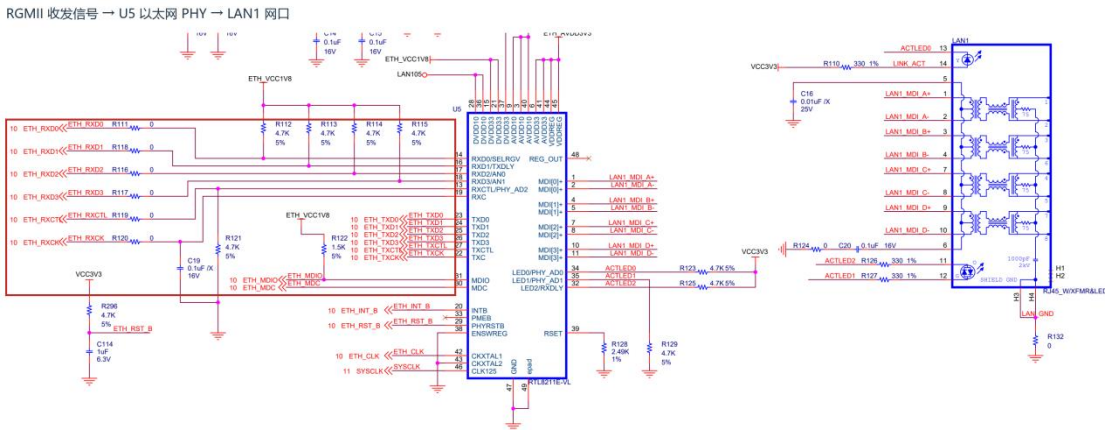


图 2-6　板载以太网 PHY 与网口电路局部

表 2-5　以太网关键连接

信号	方向（相对 PS）	作用
ETH_TXD[3:0] 、 TXCTL 、 TXCK	输出至 PHY	发送数据、控制与时钟
ETH_RXD[3:0] 、 RXCTL 、 RXCK	PHY 输入 PS	接收数据、控制与时钟
ETH_MDC / ETH_MDIO	时钟输出 / 双向	PHY 寄存器管理
ETH_RST_B	输出至 PHY	低有效复位

ARM 端运行 lwIP 协议栈并提供 TCP 服务，接收图像及运行指令，返回检测框、目标编号和处理状态。以太网接口位于 PS 固定外设通路，PL 继续承担网络算子计算。连续图像按场景批次传输，在同一批次内保持追踪状态，并利用帧号关联输入与返回结果。

（5）SD 卡启动与存储模块

SD 卡承担系统启动与数据存储两项任务。BOOT.bin 包含 FSBL、PL 位流及 ARM 应用，板卡选择 SD 启动后，由启动流程完成初始化、逻辑配置与应用加载。模型权重和测试图像以文件形式保存，ARM 通过文件系统读取；电脑传图模式下，SD 卡仍用于启动及权重加载（见图 2-7）（见表 2-6）。

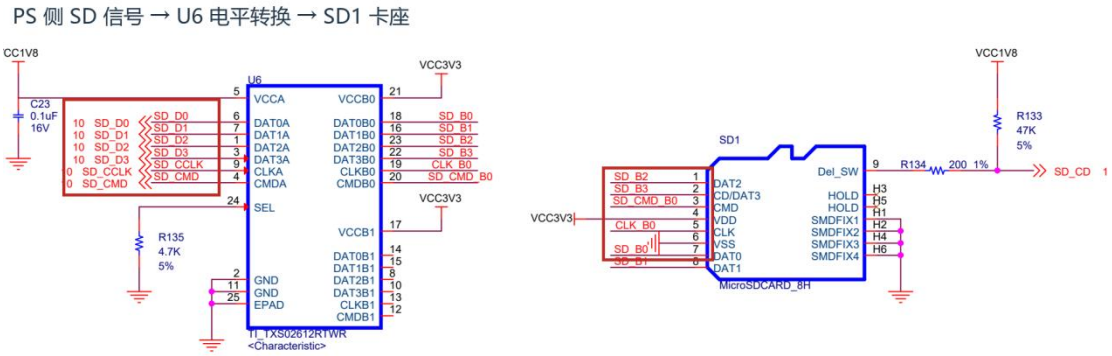


图 2-7　SD 接口电平转换与卡座电路局部 （据文献[9]原理图第 5 页）

原理图中，SD 信号通过 U6 电平转换器连接 SD1 卡座。SD_CCLK 提供时钟，SD_CMD 传输命令与响应，SD_D0～SD_D3 组成四位数据总线，SD_CD 用于插卡检测。PS 初始化参数与板卡引脚连接共同确定接口工作方式[9]。

表 2-6　SD 卡文件与接口用途

项目	内容或信号	作用
启动镜像	BOOT.bin	初始化 PS、配置 PL 并启动 ARM 程序
模型文件	folded_weights	保存部署所需网络参数
输入清单与图像	manifest.csv、sequences	关联场景、帧号和输入文件
控制与数据	CLK、CMD、DAT[3:0]	完成命令交互和数据读写

板端与电脑使用配套的场景清单，保证图像、帧号及显示结果对应。SD 图片输入和以太网图片输入最终进入相同的板端计算流程；输入来源的切换不改变 PL 加速器的计算职责。

（6）时钟、复位与供电模块

PS 通过 FCLK_CLK0 向 conv_0 和 AXI 互连提供时钟；FCLK_RESET0_N 接入 Processor System Reset，模块输出 peripheral_aresetn 连接加速器 ap_rst_n 及互连 ARESETN。图 2-8 依据工程连接关系绘制。交付 PL 实现报告的时钟约束为 120 MHz，当前以太网 FSBL 的 PS 初始化参数将 FCLK0 设置为 100 MHz，两项分别对应实现约束和启动时钟配置。



图 2-8　PL 时钟与复位信号关系

板卡电源电路将输入电源转换为不同器件所需的电压。TUL 原理图中，U18 提供 VCC1V0、VCC3V3、VCC1V8 和 VCC1V5 等电源轨，分别服务于核心逻辑、接口及存储相关电路；电源正常状态信号参与启动复位控制。 板载电源电见图 2-9。

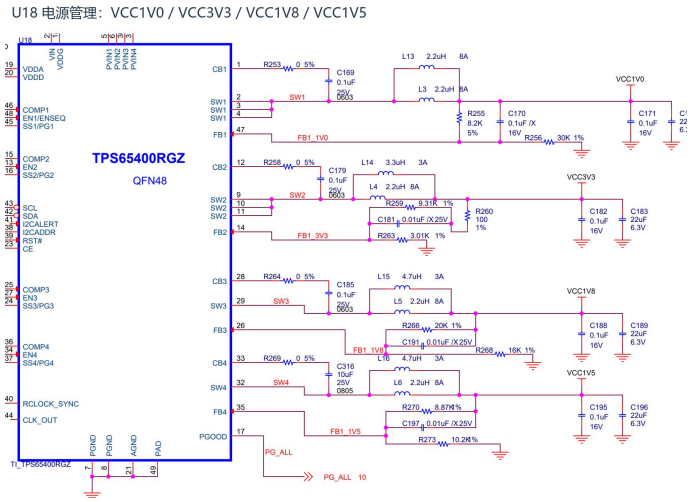
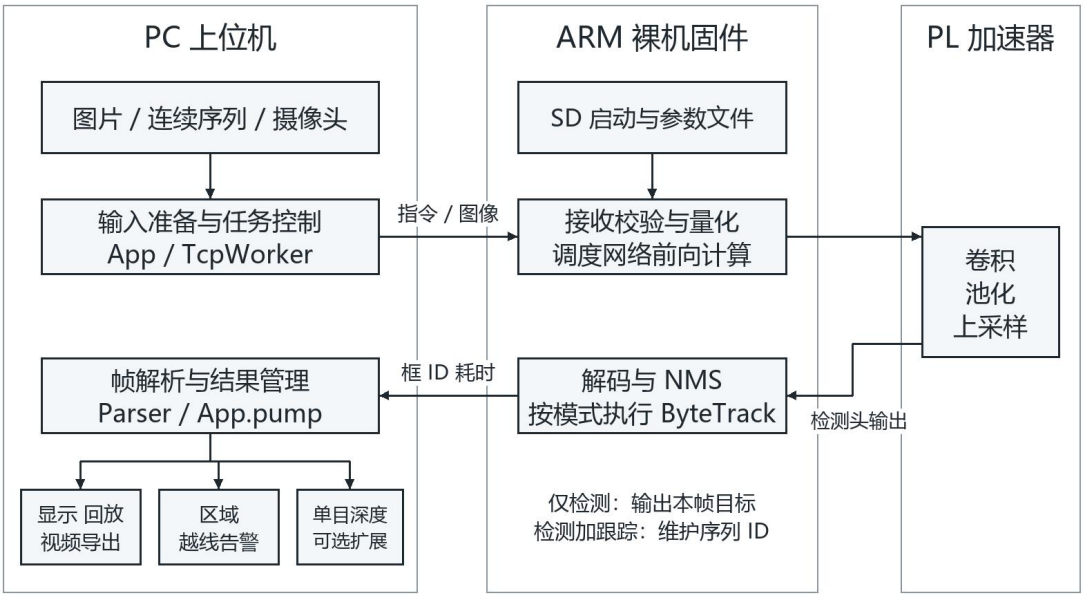


图 2-9　板载电源管理电路局部

2.3 软件系统介绍

2.3.1 软件整体介绍

海云智视采用 PC 上位机与 Zynq-7020 板端固件协同的软件架构。上位机负责图像输入、任务控制、结果显示、事件告警和单目深度扩展；ARM 裸机程序负责输入接收、网络调度、检测后处理及 ByteTrack 关联，并调用 PL 加速器完成网络中的主要计算。各模块通过运行编号、场景编号和帧号组织数据，使原图、检测框、目标 ID、耗时及告警快照能够对应到同一次处理（见图 2-10）。



以运行编号、场景编号和帧号对应原图与结果；深度任务在 PC 后台执行。

图 2-10　软件执行位置与主要数据关系

系统的数据主线为“采集或读取图像—准备输入—板端检测与跟踪—结果返回—PC 显示与告警”。仅检测模式在得到检测框后返回结果；检测加跟踪模式继续维护同一序列中的目标身份。深度预测由 PC 后台线程按需执行，读取同一原图及目标框，不改变板端检测与跟踪的任务顺序。

上位机提供图片、连续序列和摄像头三类入口。图片与序列按清单管理，摄像头则把实时预览与板卡返回结果分开显示：预览更新使用最新采集画面，检测框叠加在实际送检的对应帧上。



图 2-11　以太网上位机的真实检测结果界面

图 2-11 展示系统实际运行界面，可看到以太网连接、数据源选择、检测模式、目标列表和视频导出入口。软件界面围绕任务操作与结果复核组织：操作区发起或停止任务，主视图区显示船框及轨迹，事件列表支持定位回看。后续各模块按这条处理主线展开。

2.3.2 软件各模块介绍

（1）任务控制与运行状态管理

主控模块首先解决任务能否在正确状态下启动的问题。PC 端由 App.start()读取输入来源和运行模式，再向传输线程提交任务；ARM 端的 main()进入

run_serial_session()，完成 SD、模型参数和加速器初始化后发布 READY。该函数沿用历史串口命名，当前数据收发由 board_transport.cpp 中的以太网实现承担。

连续序列采用 BATCH 建立会话，板端保存 run_id、sequence_id、mode 及预期帧数，并重置该序列的跟踪器。每个 UPFRAME 命令先检查运行身份和帧序，再进入 RECEIVING；输入长度和 CRC 通过后进入 RUNNING，执行检测、可选跟踪和结果回传，随后回到 BATCH 等待下一帧。全部帧完成后由 BATCH_END 收尾。非法命令返回 NACK，不被当作有效任务执行（见图 2-12）。

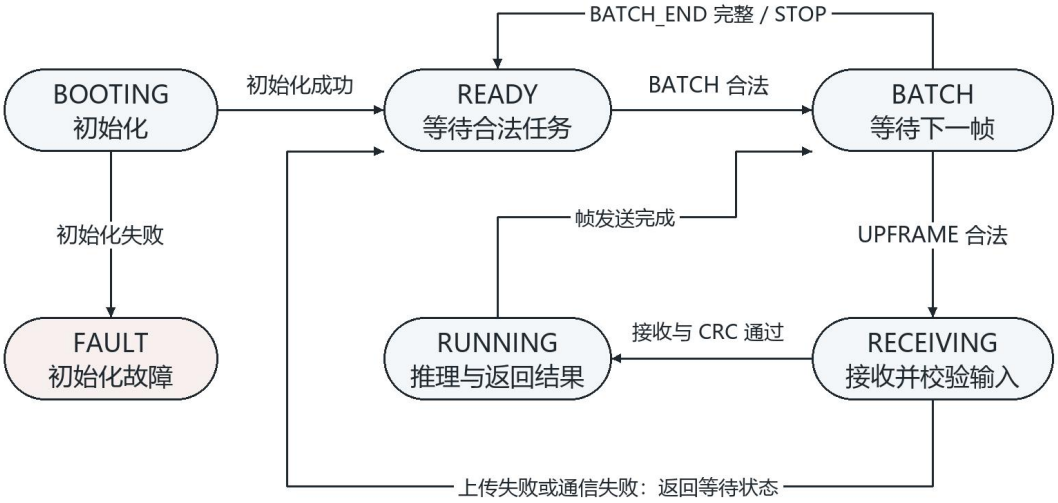


图 2-12　连续上传任务的状态转换

停止操作按任务边界处理。PC 停止继续提交新帧；当前帧的硬件计算不被立即强制中断。板端完成必要的响应后返回等待状态，并给出已处理帧数。初始化失败会进入 FAULT；连续上传中的接收或通信错误由对应异常分支返回等待状态并报告错误，避免界面把失败误显示为正常完成。

PC 端使用事件队列连接后台线程与界面。TcpWorker 负责收发，App.pump() 在主线程读取连接、结果和错误事件，再更新控件；告警和深度任务各自运行在后台。关键输入是任务模式、场景与帧清单，输出是任务状态和带身份信息的完整结果帧。

（2）图像输入准备与以太网通信

输入准备统一由 prepare_image()和 prepare_pil_image()完成：原图转换为 RGB，采用双三次插值直接缩放到 416×416，再由 HWC 排列转为 CHW 连续 uint8 字节。同时保存原图 width、height，供板端恢复横纵坐标。单帧图像载荷为 519168 字节。ARM 接收后按缩放系数 512 将像素量化为网络定点输入，与 SD 输入路径汇入同一推理接口。

摄像头由 CameraWorker 持续采集并更新 latest_rgb。App._send_camera_frame()仅在会话有效且没有在途帧时选取最新画面，保存实际上传的原图，并提交一次 UPFRAME；旧预览帧不会无限排队。因此系统允许预览与计算异步进行，显示结果始终依据运行编号和帧号关联。

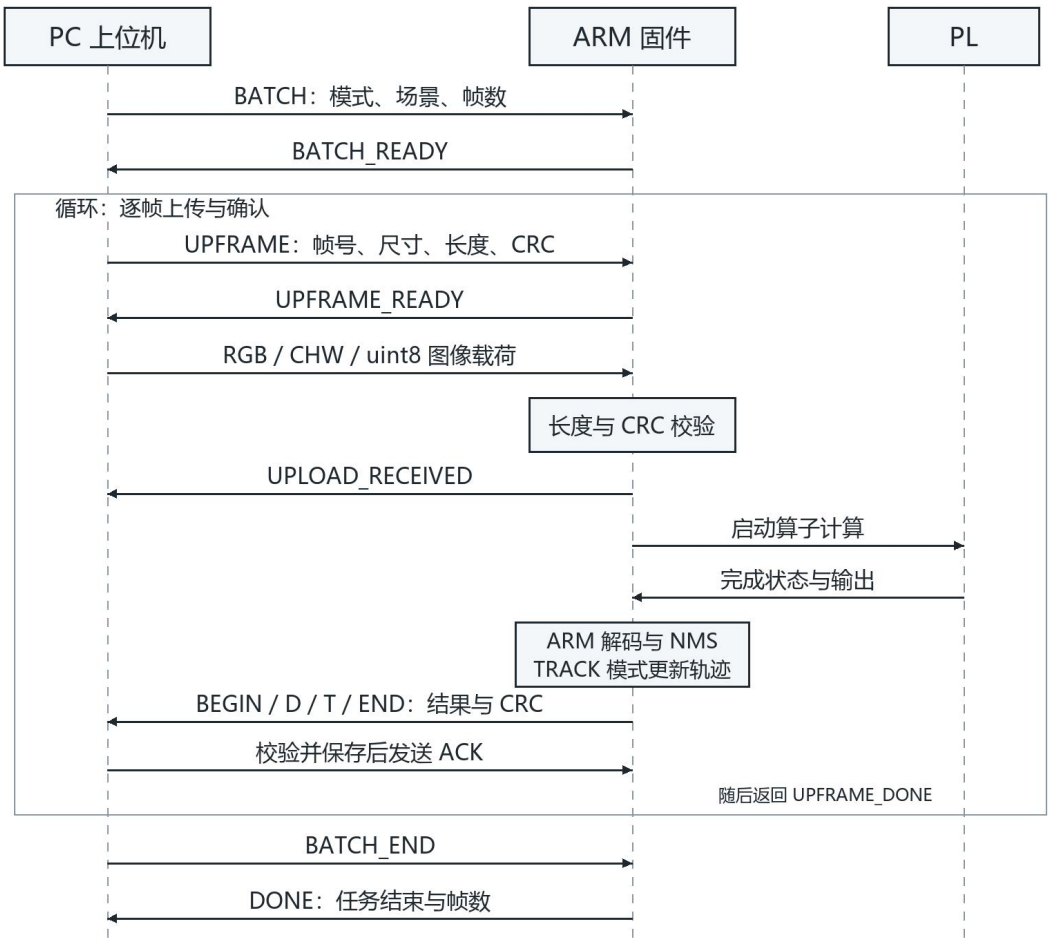


图 2-13　连续帧上传与结果确认的交互时序

图 2-13 区分了输入接收和结果确认。UPFRAME_READY 表示板端已准备接收二进制载荷；UPLOAD_RECEIVED 表示图像接收与校验完成，随后执行推理。推理结果按 BEGIN、D、T、END 封装，其中 D 为检测记录、T 为轨迹记录。PC 通过 Parser.feed()校验帧身份、记录数量、坐标与 CRC，保存完整帧后发送 ACK，板端再报告本帧结束（见表 2-7）。

表 2-7 任务与结果的关键接口数据

数据类别	关键字段	含义与使用位置
任务身份	run_id、sequence_id、frame_id	共同标识一帧；关联原图、板端结果与后续事件
输入图像	payload、width、height、crc32	uint8 CHW 图像、原图尺寸及输入完整性校验
检测记录	class_id、score、xyxy	六类编号、置信度、原图坐标系检测框
跟踪记录	track_id、detection_index、display_box	轨迹身份、匹配检测索引及实际显示框
阶段计时	timing_us	读取或接收、推理、解码 NMS、跟踪等耗时，单位微秒

传输线程仅接收本次 PC 会话认可的运行结果。相同帧的重复消息不重复写入；同一身份但内容发生变化时报告错误。无效或不完整的帧不进入绘制、跟踪结果回放和告警处理，从接口处阻止错误数据继续传播。

（3）YOLOv4-Tiny 网络框架与船舶任务适配

板端采用 YOLOv4-Tiny[1]单阶段检测网络，以 CSPDarknet-Tiny 提取多尺度特征。主干由卷积、下采样和分支拼接组成，在降低特征处理规模的同时保留不同层次的信息。BasicConv 将卷积和 LeakyReLU 组织为基本计算单元；激活函数在负半轴保留较小斜率，使负响应仍可参与后续特征表达。

图 2-14 对应板端 Yolo4_Tiny::forward()的实际计算分支。主干输出 26×26×256 浅层特征与 13×13×512 深层特征。深层特征经过 1×1 卷积得到 P5，一路直接进入 13×13 检测头；另一路先降至 128 通道并进行 2 倍上采样，再与浅

层特征拼接成 26×26×384 特征，进入 26×26 检测头。两个检测头分别覆盖步长 32 和 16 的特征尺度，高分辨率分支保留更细的位置结构。

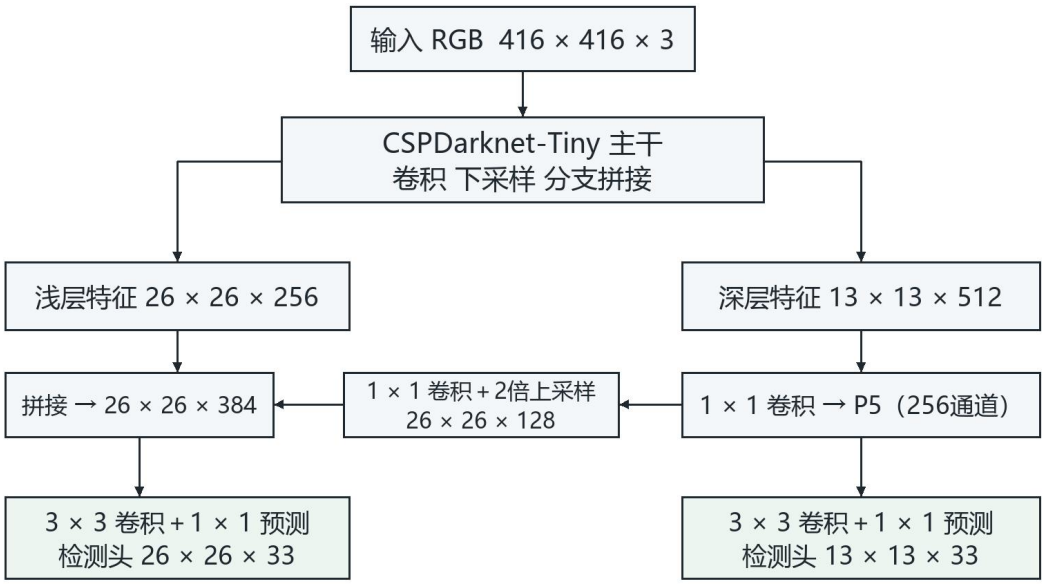


图 2-14　YOLOv4-Tiny 双尺度检测网络与实际输出分支

网络输入为 416×416×3，两个输出按空间尺寸表示为 13×13×33 和 26×26×33，在板端按 CHW 连续存储。每个网格设置 3 个 anchor，每个 anchor 包含 4 个位置参数、1 个目标分数及 6 个类别分数，输出通道数因此为 3×(4+1+6)=33，总计产生 2535 个候选预测。类别编号依次为 0 散货船、1 渔船、2 普通货船、3 矿砂船、4 客船、5 集装箱船，训练、解码与界面显示使用同一顺序。

网络前向计算与检测后处理具有明确边界：PL 相关算子输出两个定点检测头，ARM 根据 anchor 与网格位置还原候选框，再计算类别置信度和执行 NMS。网络结构、权重、类别顺序、输出布局及定点缩放系数必须保持一致。

（4）ARM 推理调度与检测后处理

网络调用与算子驱动

单帧运行函数准备 input、o0、o1 缓冲区后调用 model->forward()。Yolo4_Tiny 按网络顺序调用主干、检测头及上采样，BasicConv::forward()进一步调用 conv_leakyrelu()；采样操作调用 sampling()。底层驱动将输入、权重、偏置和输

出的 DDR 地址，以及通道数、特征图尺寸、卷积核和步长写入 AXI 控制寄存器，再调用 XConv_Start()并轮询 XConv_IsDone()。由此形成“任务函数—网络层—算子接口—硬件驱动”的调用层级。

ARM 负责中间缓冲区和层间数据衔接，PL 负责具体计算。驱动包含单算子超时检查，推理边界保留缓存维护和 XTime 计时。o0、o1 分别包含 5577 与 22308 个有符号 16 位数据，解码前检查长度。

从检测头到船舶边界框

yolo_decode_nms()接收两个检测头和 YoloPostConfig，decode_head()按 CHW 布局读取数据，先除以 512 还原实数。anchor 提供预设宽高参照，当前六组尺寸为(10,14)、(23,27)、(37,58)、(81,82)、(135,169)、(344,319)，以 416×416 输入坐标为单位。网格偏移经 Sigmoid 变换确定框中心，宽高由指数变换与 anchor 共同确定。以横坐标和宽度为例：

$$b_x=\frac{\sigma(t_x)+g_x}{G},\quad b_w=\frac{a_w\cdot exp(t_w)}{416}$$

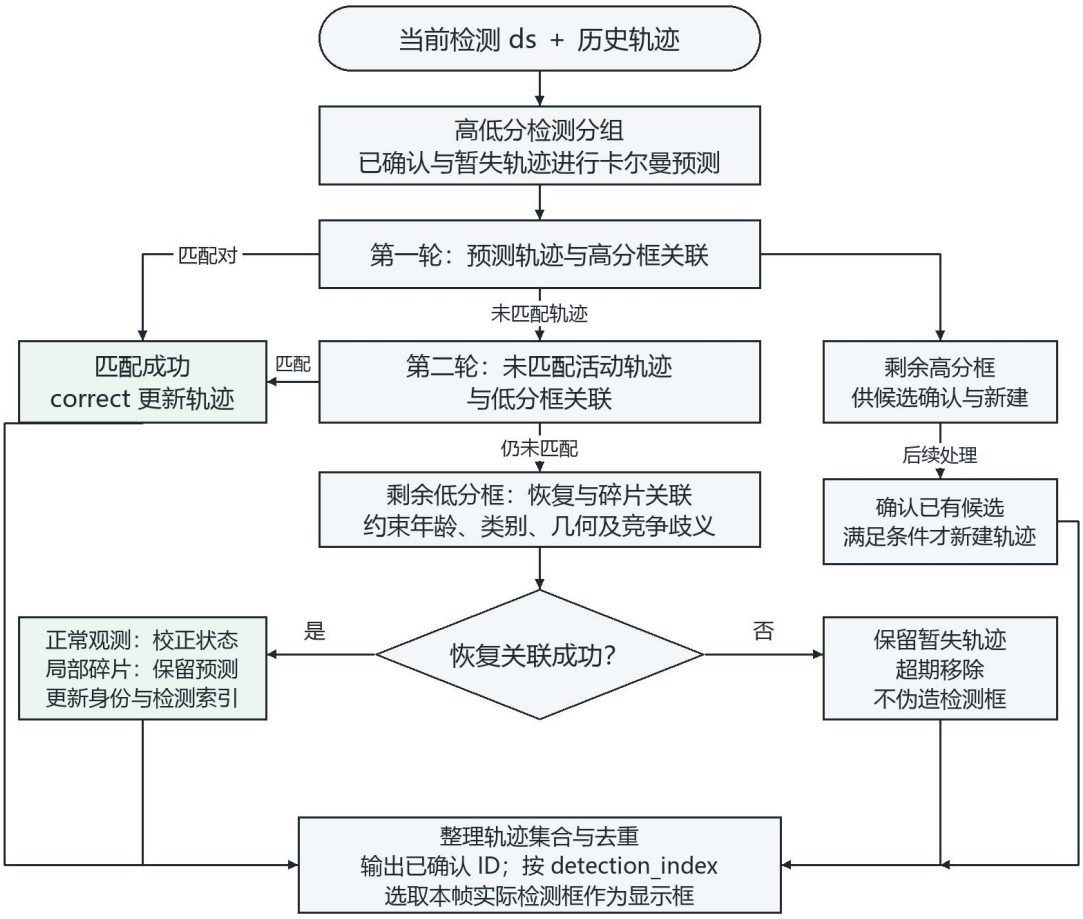
其中 G 为对应检测头的网格边长，gx 为网格列索引，aw 为 anchor 宽度；纵坐标和高度按同样方法计算。归一化坐标随后按原图宽高分别恢复为 xyxy。程序选取每个候选框得分最高的类别，将目标存在概率与该类别概率相乘作为 score。当前置信度阈值为 0.1，保留下来的低分检测还可供 ByteTrack 第二轮关联使用。

NMS 按 score 降序排列候选框，同分时按 candidate_id 保持确定顺序。程序仅比较同类别候选，在未裁剪框上计算 IoU，达到 0.3 阈值时抑制低分重复框，最终才裁剪至图像边界。输出 YoloDetection 包含坐标、分数、类别与候选索引；DET 模式直接返回检测结果，TRACK 模式将该列表传入追踪器。非有限值、非法尺寸或长度不一致返回错误。

（5）ByteTrack 关联与连续目标身份维护

ByteTrack[2]位于检测之后，输入是本帧检测列表 ds 和历史轨迹，输出是已确认目标的 ID 及对应检测关系。ArmByteTracker::update()为每条轨迹保存八维状态，包括中心位置、宽高比、高度及各自逐帧变化量，同时维护 8×8 协方差矩

阵。predict()利用恒速模型估计当前状态，correct()根据匹配检测修正预测；速度量用于图像关联（见图 2-15）。



恢复关联使用本帧检测，不增加 YOLO 推理；场景切换时 reset 清空历史。

图 2-15　高低分关联与船舶遮挡恢复的处理分支

第一轮将高分检测与已确认轨迹、暂失轨迹的预测框进行关联；未匹配的活动轨迹再与低分检测进行第二轮关联。assign()构建 IoU 代价，在指定阶段融合检测分数，并加入类别不一致惩罚，采用带未匹配虚拟项的最小代价分配。低分检测用于延续已有目标，减少弱响应造成的轨迹中断和虚假新目标（见表 2-8）。

表 2-8　当前跟踪实现的关键关联与状态参数

参数	当前配置	设计作用
检测分组	高分>0.5；低分在 0.1 与 0.5 之间	先关联较可靠检测，再尝试弱响应目标
常规新建阈值	score≥0.6	限制未匹配检测直接产生新轨迹
类别不一致代价	增加 0.1	兼顾类别信息与偶发分类波动
暂失保留	超过 30 帧进入移除处理	为短时未匹配保留原 ID 恢复机会
碎片尺寸判断	宽比<0.75 或高比<0.70	恢复关联后避免局部船体引起状态突缩

针对细长船体及遮挡场景，实现增加保守恢复与碎片关联分支。恢复只使用本帧剩余检测，综合近期轨迹年龄、类别、尺寸、位置、覆盖关系和竞争歧义筛选配对，不额外调用一次 YOLO。若恢复匹配到的只是局部船体，则保留内部预测状态、更新轨迹身份和检测索引。仍未匹配的轨迹按暂失与超期规则处理。

显示框与内部跟踪框分工不同。matched_display_box()按 detection_index 取本帧实际检测框，用于界面绘制；卡尔曼内部状态继续用于后续关联。这样可以减少平滑框滞后或过窄对展示的影响，但不会补造检测器未检出的船体。未确认候选需继续匹配确认；单一高分目标的弱初始化另有受限确认分支。切换场景或开始新批次时调用 reset()，防止不同场景共享历史 ID。

（6）结果显示与图像空间告警

PC 收到完整结果后，App.show()读取对应原图并调用 render()叠加船舶类别、分数、显示框和轨迹。历史轨迹以(sequence_id, track_id)为键记录框底边中心点；场景切换时清理历史，避免相同数字 ID 跨场景连线。结果回放直接读取已保存帧，导出视频按选定帧率重绘，并重新解码核对帧数。回放帧率描述展示速度，不代替板端实际处理速度。

告警以图像规则为基础。用户通过鼠标拖动绘制矩形警戒区或指定越线端点，软件将屏幕位置转换为原图归一化坐标。AlarmWorker 调用 Session.process()，由 AlarmEngine.process()逐帧更新规则状态，再由 Session.persist()保存事件及截图。

目标位置取检测或显示框的底边中心，以较接近船体接触水面的位置作为二维代理点（见图 2-16）。

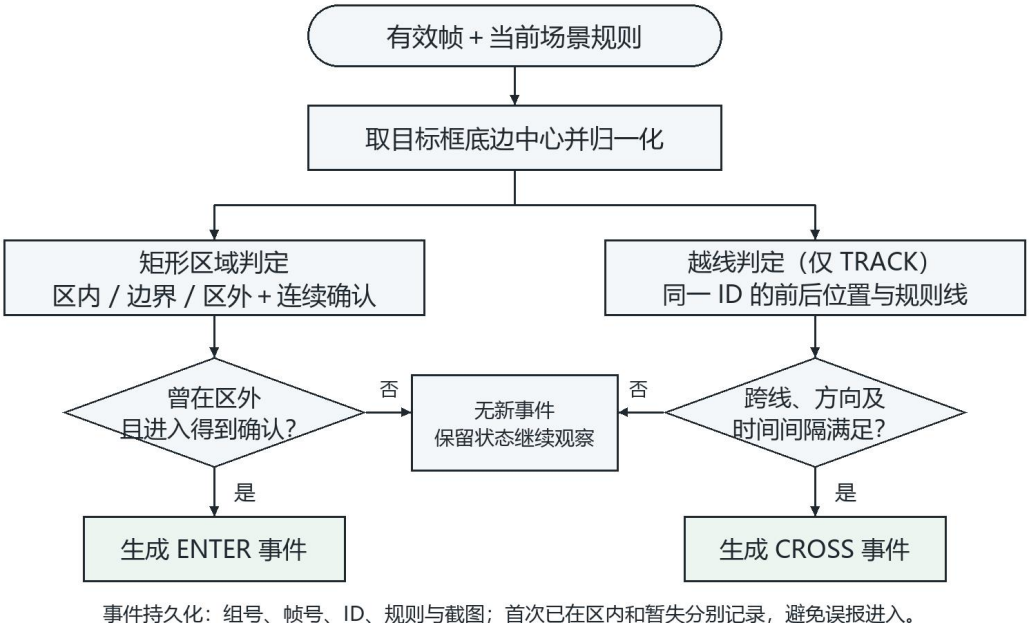


图 2-16　进入区域与定向越线的关键判定条件

区域判断区分区内、边界和区外，并使用连续帧确认减小边缘抖动。首次看到目标时若已在区域内部，记录“初始已在区内”；只有观察到区外状态后再确认进入，才生成 ENTER。短时漏检进入暂失管理，超过容限记录 LOST。仅检测模式可提示区域内检测到船。

定向越线仅在 TRACK 模式下比较同一 ID 的前后位置。算法同时检查目标位于线的两侧、两次观测之间的运动线段与有限警戒线相交、方向符合规则，以及跨帧或采集时间间隔满足要求。满足条件生成 CROSS，事件定位到对应触发帧，并保留前一观测信息用于复核。



图 2-17　G029 历史实板结果触发入区事件后的证据快照

图 2-17 来自已保存的真实告警事件，用于展示“目标结果—规则判定—事件记录—回看证据”的关系。矩形和警戒线是该次演示人工设置的图像规则。告警记录包含场景、帧号、目标 ID、事件类型和快照路径；单击事件即可回到触发画面（见表 2-9）。

表 2-9　上位机结果与事件的留存关系

保存内容	关键输出	用途
原始板端结果	frames.jsonl	保存检测、轨迹、阶段耗时与帧身份
告警事件	events.jsonl、events.csv	记录规则触发及目标、帧号，支持查询与复核
规则与证据	config_snapshot.json、截图	保留当次规则版本和触发画面
结果视频	叠加结果 MP4	按指定回放帧率展示连续检测与跟踪结果

（7）单目深度预测扩展

单目深度作为 PC 端可选后处理，与检测追踪共享原图和目标框。App.queue_range()提交图像路径、帧身份及目标信息，RangeWorker 在后台调用

RangeService.enrich()；模型延迟加载并按设备条件使用 GPU 或 CPU。后台返回 generation 与帧键，主线程核对后才更新，防止旧任务覆盖新的场景结果。

当前采用 Depth Anything V2[3]的 ViT-S 米制深度结构，输入尺度设置为 518，最大模型深度为 80m。MonocularDepthRange._predict()对整张图推理一次，将深度结果恢复到原图尺寸并缓存；measure()针对每个船框统计深度，不为同一图中的每艘船重复运行网络。框内 ROI 左右各收缩 15%、上部收缩 30%、下部收缩 8%，以减少框边天空、背景和水面的影响（见图 2-18）。

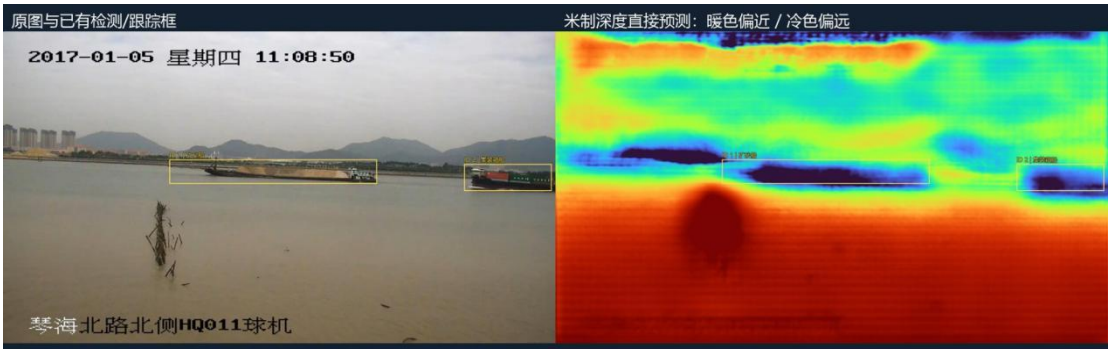


图 2-18　同一 G029 图像的目标框与单目深度预测对照

ROI 按步长 2 采样，剔除非有限值和 1～80m 以外的值；有效像素不足 20 个时返回无效原因。有效时以中位数估计相机前向 Z 深度，结合 P10、P90 描述框内离散程度。关键输出为 distance_m、valid、reason 及深度图；其中辅助质量分数反映分布稳定程度。

第三章 完成情况及性能参数

3.1 整体介绍

本系统已完成 PYNQ-Z2 板端部署与电脑上位机联调，形成图像输入、船舶检测、连续追踪、规则告警和结果回看的完整流程。电脑图片与摄像头画面通过以太网送入板卡，SD 卡保留启动、权重及场景图像存储功能。板端返回目标类别、位置、编号与处理状态，电脑端完成显示、交互告警及单目深度预测。

3.2 工程成果

3.2.1 硬件成果

硬件平台采用 Zynq-7020 器件，已完成 HLS 加速模块封装、PS/PL 互连、综合实现及启动镜像制作。PL 承担网络算子计算，PS 负责逐层调度、解码、非极大值抑制与 ByteTrack 关联，并通过以太网与电脑交换数据。板载 DDR 支撑输入、权重和中间特征存储（见图 3-1）。

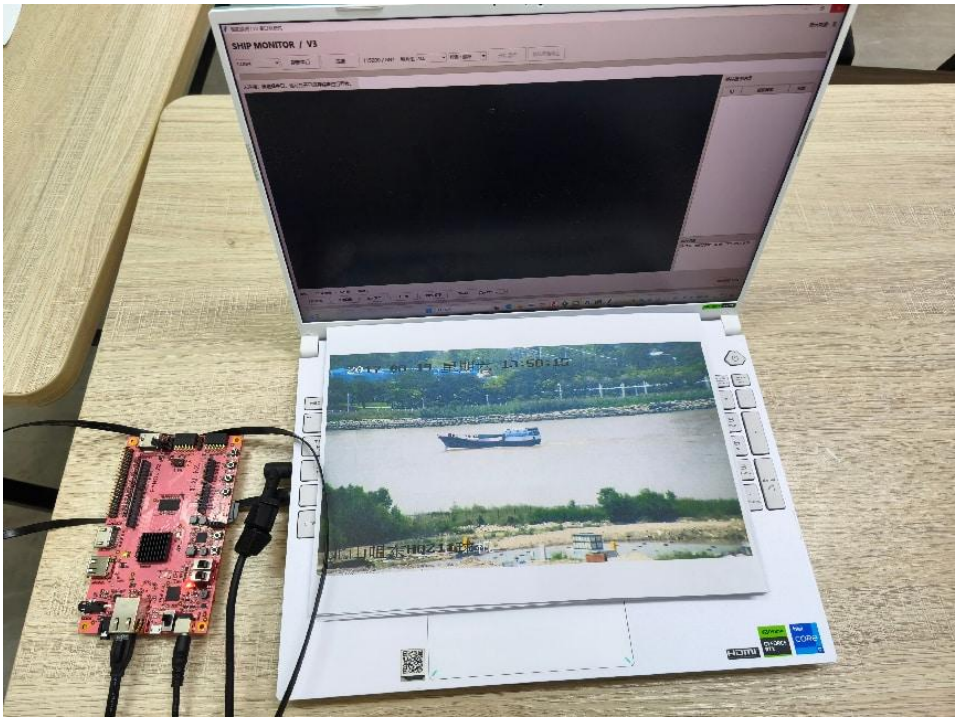


图 3-1　系统采用的 PYNQ-Z2 硬件平台

3.2.2 软件成果

上位机提供独立图片、连续场景及摄像头输入，支持仅检测、检测加追踪、检测加追踪加测距三种模式。运行结果集中展示检测框、类别、置信度、目标编号和轨迹；用户可设置警戒区或越线规则，并通过事件记录定位触发画面。结果保存、回放和导出功能用于核验与展示（见图 3-2）。



图 3-2　以太网上位机实际运行界面

3.3 特性成果与性能测试

功能验证与量化测试分别覆盖模型检测、跨帧关联、板端处理、告警事件及电脑端深度预测。各项结果使用对应日志或评估文件统计，测试对象与样本范围见表 3-1。

表 3-1　测试对象与数据范围

测试内容	样本范围	评价对象
船舶检测	3500 张图像、4683 个目标	PC 模型复评
连续追踪	17 组 189 帧、28 个真实 ID	PC 回放关联结果
板端检测精度	500 张图像、667 个目标	同批 PC 与板端对照
实板运行	7 组 85 帧	SD 输入检测加追踪
摄像头输入	16 帧	以太网接收与板端处理
深度模块	61 帧性能测试；497 帧精度测试	GPU 运行开销与 SeePerSea 真值评估

3.3.1 船舶检测效果

（1）测试数据与评价设置

检测实验使用 SeaShips[5]数据中的 3500 张图像，共包含 4683 个标注目标，覆盖散货船、渔船、普通货船、矿砂船、客船和集装箱船六类。评价采用经核对的 XML 标注。YOLOv4-Tiny 与改进 YOLOv5 的输入尺寸均为 416×416，NMS 的 IoU 阈值设为 0.3；Precision、Recall 与 F1 在置信度阈值 0.25、匹配 IoU 阈值 0.5 下统计。mAP 采用置信度下限 0.001 的预测结果计算，mAP50:95 按 IoU 从 0.50 至 0.95、步长 0.05 统计。

YOLOv4-Tiny 采用 RGB 图像双三次直接缩放，改进 YOLOv5 采用双线性缩放与方形填充，各自保持对应模型的推理预处理。实验比较现有训练权重在同一批图像上的检测表现，用于部署模型选择（见表 3-2）。

（2）模型对比与部署选择

表 3-2　两种检测模型在 3500 张图像上的评价结果

模型	mAP50（%）	P（%）	R（%）	F1（%）	漏检数
YOLOv4-Tiny	93.95	91.77	90.26	91.01	456
改进 YOLOv5	82.35	87.18	75.51	80.92	1147

YOLOv4-Tiny 的 mAP50、Recall 和 F1 分别达到 93.95%、90.26%和 91.01%，较改进 YOLOv5 分别提高 11.60、14.76 和 10.09 个百分点；漏检数量由 1147 个降至 456 个，减少 60.24%。两者 mAP50:95 分别为 55.72%和 55.13%。结合船舶持续检出对后续目标关联和规则告警的作用，本系统选择 YOLOv4-Tiny 完成板端部署（见表 3-3）。

（3）分类别检测表现

表 3-3　YOLOv4-Tiny 六类别检测结果

船舶类别	目标数	P（%）	R（%）	AP50（%）
散货船	995	91.85	90.55	95.05
渔船	1188	90.75	85.10	90.07
普通货船	744	93.31	95.56	96.99
矿砂船	1071	89.21	91.13	93.94
客船	248	92.27	81.85	89.45
集装箱船	437	97.70	97.25	98.22

分类别结果用于观察不同船型的检出差异。渔船召回率为 85.10%，低于总体召回率，说明该类别仍有较多漏检；普通货船召回率为 95.56%，检出表现较好。图 3-3 给出独立图像经板端处理后的实际输出，检测框、类别与置信度共同体现系统的分类与定位功能。



图 3-3　独立图像的实板检测结果

（4）板端检测精度验证

为核验硬件部署后的检测表现，从上述 3500 张图像中按固定随机种子 20261104 抽取 500 张，共包含 667 个标注目标，覆盖六类船舶。输入图像转换为 416×416、RGB 通道、CHW 排列的 float32 数据，由板卡从 SD 卡读取，在仅

检测模式下完成 FPGA 推理及 ARM 解码、NMS，再通过以太网回传结果。测试共完成 500 帧，运行状态为 SUCCESS，回传帧号与原图逐一对应。

PC 与板端采用相同 500 张图像及 XML 标注，在置信度阈值 0.25、匹配 IoU 阈值 0.5 下进行分类别、一对一目标匹配。当前板端输出保留置信度不低于 0.1 的检测框，本项对照采用固定阈值下的 Precision、Recall、F1 及目标计数，结果如表 3-4 所示。

表 3-4　同 500 张图像的 PC 与板端检测精度对比

评价指标	PC 模型	PYNQ-Z2 板端
Precision（%）	91.10	91.36
Recall（%）	89.06	88.76
F1（%）	90.07	90.04
正确检出 TP（个）	594	592
误检 FP（个）	58	56
漏检 FN（个）	73	75

板端 Precision 为 91.36%、Recall 为 88.76%、F1 为 90.04%。与同批 PC 结果相比，Precision 提高约 0.25 个百分点，Recall 下降约 0.30 个百分点，F1 下降约 0.03 个百分点；正确检出减少 2 个，误检减少 2 个，漏检增加 2 个。结果表明，在本次 500 张图像上，模型完成板端部署后保持了与 PC 参考接近的整体检测表现。

3.3.2 连续目标追踪效果

连续追踪评估采用 17 组 189 帧人工标注序列，包含 303 个逐帧目标实例和 28 个组内真实目标 ID。将板端输出的追踪框及目标编号与人工标注进行匹配，计算 IDF1、MOTA、ID 切换次数和轨迹中断次数。匹配 IoU 阈值为 0.5，匹配过程不区分类别（见表 3-5）。

表 3-5　连续追踪评估结果

指标	结果	评价重点
IDF1	86.16%	身份关联的综合表现
MOTA	72.61%	漏匹配、误匹配与 ID 切换
IDSW	2 次	真实目标身份切换次数
Frag	4 次	轨迹中断次数



图 3-4　G029 实板追踪结果：上为第 1 帧，下为第 12 帧

图 3-4 中，两艘船在不同位置仍保留 ID 1 与 ID 2，轨迹反映其运动方向。代表性短序列 G029、G078、G094、G017 在评估中均无 ID 切换和漏匹配。

3.3.3 区域告警与事件回看

告警功能以图像区域和越线规则为依据，将目标位置、身份编号与事件记录关联。用户先在场景画面中设置规则，再运行检测与追踪；满足规则后，上位机保存事件类型、触发帧、目标 ID 及快照，并支持点击记录定位回看（见图 3-5）（见表 3-6）。



图 3-5　G066 目标进入警戒区时保存的实际事件快照

表 3-6　告警事件与帧记录核验

核验项	结果
场景与处理范围	G066，共 12 帧
入区事件	1 次，日志 frame_id=7，目标 ID 1
观察结束记录	1 条，日志 frame_id=11
事件快照	2 张，均可对应事件记录
帧号与原图对应	2 条事件均匹配运行帧记录

本组记录来自以太网工程运行日志。快照保留目标框与警戒区，事件记录保留触发帧及原图文件名，形成从监测结果到事件追溯的关联。表中数量描述本次运行的事件输出，评价重点是规则触发与记录对应关系。

3.3.4 单目深度预测效果

单目深度功能在电脑端执行，将输入图像转换为深度分布，并与船舶检测结果关联显示。图 3-6 选取场景 4 的灰暗低对比度画面和场景 5 的远处小目标画面，展示原图与对应深度图，辅助观察目标及周围场景的远近关系。界面中的距离属于模型估计值。

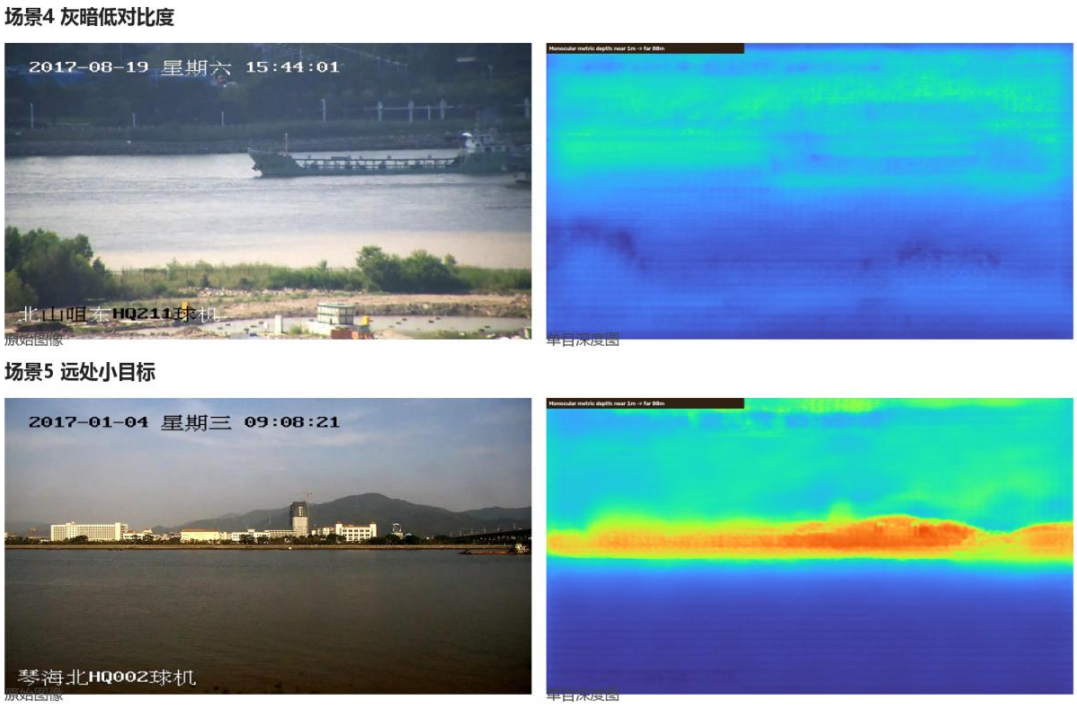


图 3-6　场景 4 与场景 5 的图像及深度预测结果

（1）SeePerSea 测试集深度精度

为评价单目深度预测的准确性，本项目采用 SeePerSea 数据集[12]中的同步图像、激光雷达点云及标定参数，将点云投影至图像平面，以相机坐标系下的前向深度作为参考真值，在有效投影位置比较模型预测值与真值。

实验采用 Depth Anything V2 Metric Depth ViT-S 模型，以 VKITTI 预训练权重为基础，冻结图像编码器并微调深度预测头。数据按视频序列划分，训练集包含 20 段视频、3435 帧，验证集包含 4 段视频、561 帧，测试集包含 7 段视频、497 帧。输入尺寸参数设为 518，batch size 为 1，学习率为 3×10^{-5}，训练 3 轮，根据验证集 MAE 选择最佳权重。评估深度范围为 1～80 m，测试集共包含 29,798 个有效深度点。

表 3-7 SeePerSea 测试集深度预测精度对比

评价指标	原始预训练模型	微调后模型
MAE（m）↓	7.0500	6.5209
RMSE（m）↓	10.5061	8.9191
AbsRel ↓	0.3199	0.1983
δ_1（%）↑	69.01	65.66

MAE 与 RMSE 反映预测深度的绝对误差，AbsRel 反映相对于真实深度的误差比例；δ_1表示预测值与真值之比及其倒数均小于 1.25 的有效点占比。微调后，MAE、RMSE 和 AbsRel 分别下降约 7.5%、15.1%和 38.0%，平均深度误差及较大偏差得到改善；δ_1下降 3.36 个百分点。

（2）深度模块处理性能

表 3-8 Depth Anything V2 Small 独立测试性能

项目	结果
测试规模	61 帧
平均处理耗时	119.7 ms/帧
P95 处理耗时	135.5 ms/帧
峰值显存	0.23 GiB
测试环境	RTX 4060 Laptop GPU；FP16；batch=1

表 3-8 为相对深度模型的独立运行性能测试：预热 3 次后统计，处理范围包括预处理、GPU 传输、模型推理、输出缩放及传回 CPU。该测试使用 61 帧图像，评价深度模块的运行开销；表 3-7 则使用 SeePerSea 保留测试集，评价米制深度模型微调前后的预测精度。

3.3.5 摄像头在线检测与追踪

摄像头采集打印船舶照片，由电脑完成图像准备并经以太网上传，板端执行检测与追踪后回传结果。与固定图片输入相比，摄像头能够持续产生新的输入；移动照片时，可观察目标位置、身份编号及轨迹随采集画面更新（见图 3-7）（见表 3-9）。



图 3-7　摄像头采集打印照片并返回板端追踪结果

表 3-9　摄像头输入 16 帧实板测试

阶段	平均耗时（ms/帧）
板端接收阶段	340.15
FPGA 推理阶段	1055.97
ARM 解码与 NMS	4.75
ARM 目标追踪	0.22
推理与后处理合计	1060.94

电脑预览画面与板端结果分别更新，右侧结果对应已完成处理的采集帧。

3.3.6 实板处理性能与 FPGA 资源占用

（1）独立图像的板端处理耗时

对同一轮 500 张仅检测测试的逐帧计时记录求算术平均值，结果见表 3-10。输入读取、FPGA 推理与 ARM 后处理的合计平均为 1350.11 ms/帧，其中推理阶段约占 76.73%。本轮不执行目标追踪；合计不包含 SD 结果写入、以太网结果回传和电脑绘图。

表 3-10　500 张仅检测模式的实板处理耗时

处理阶段	平均耗时（ms/帧）
SD 输入读取	309.58
FPGA 推理阶段	1035.87
ARM 解码与 NMS	4.66
板端读取、推理与后处理合计	1350.11
SD 结果 CSV 写入（单独统计）	4.72

500 张图像的上述板端合计耗时累计约 675.05 s，即 11.25 min。该值为逐帧计时之和，未计入启动、加载权重及逐帧通信等待等开销。与后续 85 帧检测加追踪测试分别列示，用于对应不同运行模式及输入样本。

（2）连续场景的实板处理耗时

检测加追踪模式下，对 G066、G029、G071、G078、G002、G138 和 G094 七组场景连续运行，共处理 85 帧。板端完成 85 帧，电脑接收 85 帧，七组均返回 SUCCESS。表 3-11 由全部 85 帧日志计算算术平均值。

表 3-11　SD 输入 85 帧实板处理耗时

处理阶段	平均耗时（ms/帧）	占板端合计
SD 输入读取	304.95	22.98%
FPGA 推理阶段	1017.29	76.64%
ARM 解码与 NMS	4.70	0.35%
ARM 目标追踪	0.35	0.03%
板端处理合计	1327.28	100%

板端平均耗时中，推理阶段占 76.64%，输入读取占 22.98%，二者构成主要时间开销。解码、NMS 和追踪合计约 5.04 ms/帧，表明当前性能改进应优先关注网络计算及输入组织。表内分项经四舍五入，显示值之和可能与合计略有差异。

（3）器件资源与时序结果

资源数据取自最终位流对应工程的布线后实现报告，目标器件为 Zynq-7020。当前设计通过复用 HLS 计算模块完成多层网络运算，资源使用情况见表 3-12。

表 3-12　FPGA 实现资源占用

资源	使用量	器件总量	占用率
LUT	36,743	53,200	69.07%
FF	39,527	106,400	37.15%
BRAM	94.5	140	67.50%
DSP	220	220	100.00%

BRAM 按 36 Kb 块等效统计。DSP 已全部使用，反映乘加计算对器件算术资源的需求；LUT 和 BRAM 占用分别约 69%和 68%。后续扩展计算并行度时，应结合 DSP 约束、访存带宽与时序结果综合判断（见表 3-13）。

表 3-13　实现时序与启动时钟配置

项目	结果	对应范围
实现时钟周期	8.333 ns	PL 布线后约束
约束频率	约 120 MHz	对应 8.333 ns 周期
WNS / TNS	0.062 / 0.000 ns	建立时间检查
WHS / THS	0.015 / 0.000 ns	保持时间检查
时序检查结论	约束通过	实现报告
FCLK0 启动配置	100 MHz	以太网 FSBL 初始化

实现报告在所设约束下满足时序要求；当前启动配置与实现约束频率分别列示，便于对应工程构建及实际运行条件。板端耗时采用实测日志统计，不根据约束频率换算为其他频率下的推理性能。

综合测试结果，系统已完成船舶检测、连续追踪、规则告警、深度显示和摄像头输入的功能集成，并形成模型评估、运行日志、事件快照及硬件实现报告等验证材料。检测与追踪指标反映算法表现，实板耗时及资源指标反映部署开销，共同支撑系统完成情况的评价。

第四部分　总结

4.1　可扩展之处

后续优化可优先围绕卷积计算与数据传输展开。在现有 DSP 资源已用满的条件下，通过改善分块访存、缓冲复用与传输流水降低端到端时延，再评估剪枝和低精度计算的收益。应用侧可扩充真实水域中夜间、雾天和遮挡场景的数据，优化小目标检出与身份保持；结合固定相机标定和实测距离，完善单目深度的尺度校准与误差评估。系统还可接入多路摄像头，并在可靠的空间标定与运动估计

基础上扩展船舶流量统计和风险分级。通过更换模型、类别表及告警规则，整体架构可进一步适配道路车辆或园区目标监测。

4.2 心得体会

本项目围绕自主选题赛道的系统完整性、软硬件协同与可复现性要求，完成船舶检测、追踪和上位机交互，并将开发中的问题进一步整理为可复用的验证方法。研发过程使我们认识到，功能演示、性能测量和工程交接需要分别落实到具体接口、运行记录与验证步骤。

4.2.1 软硬件协同与系统联调

自主选题让我们认识到，作品的完整性取决于算法、硬件与应用能否协同工作。开发中，我们将规则性较强的卷积和采样运算放在 PL，将网络调度、检测后处理、ByteTrack 关联及通信放在 ARM，电脑负责采集、显示、告警与深度预测。这种分工使各模块能够独立调试，也让有限的硬件资源集中用于主要计算任务。模型选型同时考虑检测表现、可用算子与存储开销：YOLOv5 优化用于算法研究，实际板端采用 YOLOv4-Tiny，两者的结构与性能数据分别归档。

从模型到板端，输入尺寸、RGB 通道、CHW 排列、量化尺度、类别顺序与检测头通道数必须一致。PS 与 PL 共享 DDR，还需在硬件读取前写回缓存、处理器读取结果前使缓存失效。我们逐步形成了先核对数据契约，再定位计算错误的调试习惯，避免把格式或缓存问题误判为模型失效。

联调按启动、网口链路、TCP 连接与应用 READY 分层检查；图像上传通过长度与 CRC32 验证完整性，并以运行编号、场景和帧号对应结果。同一批次保留追踪状态，换场景重新初始化；摄像头预览与送检分开运行，减少画面积压。现有 DSP 资源已用满，后续优化应结合访存和传输耗时，不宜只追求增加计算并行度；本地预览帧率也不能替代系统检测帧率。

4.2.2 大模型协作与纠错体会

我们采用成员提出目标与约束、模型辅助分析和编写、再依据源码与运行证据复核的协作方式。模型适合梳理接口、组织测试和整理文档，但其回答仍需检

查来源与适用条件。例如，报告中的研究模型与部署模型需要分开，构建成功、仿真通过和实板运行也必须分别记录。

在大模型辅助开发过程中，我们将接口分析、模块实现、故障定位与测试记录逐步整理为可复用的 Skill。团队负责明确需求、运行条件与验收标准，模型按任务读取工程文件、提出修改方案并辅助生成代码和测试脚本。最终将操作步骤、调用示例及验证记录一并交付，使后续使用者能够依据具体工程条件开展开发与调试。

4.2.3 从具体问题提炼通用经验

团队围绕 ARM standalone C/C++、HLS C/C++加速器及 Vitis/Vivado 工具链，整理形成 zynq-baremetal-hls-validation 技能。其流程按需求与任务划分、接口设计、模块实现、联合调试、优化决策和工程交付展开：先确定 PC、ARM 与 PL 各自职责，再明确数据和控制接口，随后依据测试结果逐步完善实现。模型类别、寄存器地址和板卡配置作为工程输入。

数值正确性与内存访问安全是两类独立检查。算子测试除比较有效输出与独立参考结果外，还检查输出保护区及输入地址范围，并覆盖单 tile、多 tile、尾通道、padding、stride 和极值输入。发生差异时，分别记录算子代码、参考程序及仿真配置的修改，避免通过扩大缓冲区或改变参考结果掩盖问题。

构建与联调经验被整理为分层定位方法。编译阶段从实际工作目录解析 BSP 头文件和库路径；启动打包阶段检查 FSBL、bitstream、应用 ELF 与 BIF 的对应关系；PS 与 PL 交互阶段明确数据布局、量化、地址、缓存维护和完成条件。通信异常则沿启动、连接、应用 READY、帧接收、计算与结果返回逐级定位，通过同一帧的记录确定故障发生的位置。

4.2.4 技能结构与工具实现

技能包外层以 skill/README.md 说明用途、入口及使用方式，内部以 zynq-baremetal-hls-validation/SKILL.md 作为智能体入口。入口文件声明适用任务，并引导模型按问题读取技术说明；scripts 保存辅助工具，assets 提供接口与修复记

录模板，prompts 提供调用示例，examples 与 tests 支持脚本测试。验证记录集中保存在外层 validation 目录，各文件职责见表 4-1。

表 4-1　技能包主要文件与用途

文件或目录	用途
README.md、SKILL.md	说明使用方法；声明技能适用范围，组织开发流程并选择相应技术说明。
scripts	提供文件哈希比较、数值对照、Vivado 报告导出与报告汇总四个辅助工具。
references、assets	说明裸机构建、PS-PL 接口、HLS 测试和通信定位；提供接口契约与修复记录模板。
prompts、examples、tests	保存调用示例、数值与报告样例及脚本回归测试程序。
validation、来源及清单文件	保存本次测试记录、基础包来源与文件 SHA256 清单，明确测试对象和范围。

工具部分保留四个可直接调用的脚本：check_files.py 比较文件 SHA256，compare_values.py 按给定容差比较数值数组，export_reports.tcl 从 Vivado 检查点导出实现报告，summarize_reports.py 提取资源、时序、时钟及功耗估算。Python 脚本在电脑端运行，板端保持 C/C++实现。工具通过明确输入、输出和退出码支持重复执行，便于把模型建议落实为可检查的结果。

针对本项目的 PS 与 PL 接口，技能优先指导 ARM 通过控制寄存器配置加速器，由加速器 m_axi 接口直接访问 DDR。流程要求检查缓冲区地址、容量、生命周期及缓存一致性，并依据实际控制协议判断完成和超时；只有工程确实采用 AXI DMA 时才进入对应的驱动与流接口检查。软件后处理修改与 HLS 硬件修改分别确定构建范围，避免遗漏必要更新或重复生成无关产物。

4.2.5　适用边界与复用方式

技能适用于 Zynq 裸机 C/C++与 HLS 加速器的模块开发、接口联调、边界测试和工程交付。使用者需提供源码、平台、工具版本及真实接口定义；缺少地址、数据格式或控制协议时，先查明这些条件。现有报告解析器适配英文 7 系列资源标签，其他器件系列或报告格式需要进一步适配。

其他队伍可复制完整技能目录，阅读 README 并运行 tests/run_tests.py，再提供自己的工程路径与目标模块，通过$zynq-baremetal-hls-validation 调用。调用示例覆盖裸机模块开发、HLS 边界排查和实现报告分析，并分别明确任务范围、输入条件及交付要求。迁移时替换器件、时钟约束、寄存器定义、数据布局和测试样例，保持“先明确接口、再分层测试”的方法。

4.3 复现说明

本项目选用 Vivado/Vitis 2019.2 作为统一开发工具链，完成 HLS 加速模块集成、硬件系统构建、ARM 软件编译及板级联调。考虑到软硬件协同开发对工程配置和工具版本一致性的要求，项目在实现与验证过程中保持工具链稳定，以保证构建过程和测试结果的一致性。报告明确开发环境，并随工程提供配套配置、构建脚本与操作步骤，便于他人复现。

以下步骤对应 haiyun_fpga.rar 解压得到的 haiyun_fpga_submission 目录，采用包内启动镜像与默认 91 帧数据完成系统运行复现。路径均以项目根目录为基准，硬件平台为 PYNQ-Z2，板端运行裸机固件，电脑运行 Python 上位机。当前包内文件清单包含 3834 项。

步骤 1　准备设备与解压目录：准备 PYNQ-Z2、与板卡匹配的电源、MicroSD 卡及读卡器、网线、Windows 电脑；验证摄像头功能时另备可被电脑识别的摄像头。将压缩包完整解压到短英文路径，例如 D:/haiyun_fpga_submission。进入同时包含 README.md、start_pc.cmd、board、data、src 等项目的根目录，后续命令均在该目录执行。

步骤 2　核对文件完整性：在 Windows 命令提示符中运行 python build/verify_manifest.py。脚本依据 report/file_manifest.json 逐项检查 SHA256；应得到 checked 为 3834 且 errors 为空的输出。检查 board/sd_card 中包含 BOOT.bin、folded_weights、sequences 和 manifest.csv；确认 data/pc_files 中同时存在 manifest.csv 与 images 目录。使用默认 91 帧清单及其配套图片。

步骤 3　配置电脑软件环境：安装带 Tk 支持的 Python，在项目根目录打开 Windows 命令提示符，依次执行下列命令，建立独立环境、安装依赖并指定启动

解释器。依赖清单包含 pyserial、Pillow、OpenCV、NumPy、PyTorch、torchvision 和 matplotlib。

```
python -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
set "SHIP_PYTHON=%CD%\.venv\Scripts\python.exe"
```

步骤 4　准备 SD 启动分区：先备份 SD 卡原有内容，在板卡断电状态下，将 board/sd_card 目录中的全部内容复制到 FAT32 启动分区根目录，而不是把 sd_card 文件夹整体嵌套进去。保留权重与序列文件的相对位置。设置板卡为 SD 启动，插入 SD 卡后接通电源，等待 PL 配置和 ARM 应用初始化。已有配套 SD 卡时不必每次重新复制。

步骤 5　设置有线网络：用网线连接电脑和板卡，将电脑有线网卡 IPv4 设为 192.168.1.20、子网掩码 255.255.255.0；直连时网关与 DNS 可留空。板卡服务地址为 192.168.1.10，端口 5000。检查有线链路及本机防火墙设置，保证该网段通过有线网卡访问。

步骤 6　启动上位机并等待就绪：在已设置 SHIP_PYTHON 的命令提示符执行 start_pc.cmd；也可使用虚拟环境解释器直接运行 src/vitis/pc_app/app.py。只保留一个本工程上位机窗口。在界面填入板卡 IP 与端口后点击“连接”，等待“板卡已就绪”或 READY。按钮变成“断开”仅表示 TCP 已连接，仍应等待应用就绪后再开始推理。

步骤 7　选择数据来源与处理模式：点击“加载电脑数据”使用 data/pc_files 中的原图和清单；若弹出选择框，应选择同时包含 manifest.csv 与 images 的父目录。点击“使用 SD 卡场景”则由板卡读取 SD 输入，电脑使用 src/vitis/data 中的配套清单和图片显示结果。先选照片组，再选模式，因为切换照片组会恢复“检测＋跟踪”。不连续图片组或 ALL 组使用“仅检测”，连续照片组使用“检测＋跟踪”。

步骤 8　预览首帧并配置预警：选择连续照片组及模式后，界面自动显示首帧原图，无需先启动推理。需要区域预警时勾选“预警”，点击“画矩形区”，在原图内按住鼠标拖出矩形；每增加一个区域需再次点击绘制按钮。越线规则使用“画越线”，从 A 拖到 B。按 Esc 取消尚未完成的绘制，“清除本组”只清理当前组规则。规则保存在 src/vitis/configs，运行前检查是否适合当前画面。

步骤 9　运行单组检测或追踪：设置完成后只点击一次“开始运行”，检查返回的类别、置信度、船框、目标 ID 与当前帧是否对应。运行期间不切换照片组、数据来源或模式，也不另开回放和任务。等待“本次运行结束”后再操作。如需中断，“断开”会终止通信，重新连接并恢复 READY 后应作为新任务从头运行。

步骤 10　运行七个连续场景：待当前任务完成并恢复 READY 后，如需预警，可先在 SD 模式下逐组选取跟踪 01 至跟踪 07 并配置规则。随后点击独立的“运行场景 1-7 追踪”按钮，即开始顺序执行，不再点击“开始运行”。该入口固定采用 SD 数据与“检测＋跟踪”，不沿用电脑上传或测距模式；各场景切换时重置跟踪状态，检查不同场景的编号没有混用。

步骤 11　验证单目测距：先选择连续照片组，再选“检测＋跟踪＋测距”。确认 src/depth_anything_v2/metric_depth 源码目录与 data/models/depth_best.pth 权重存在，Python 环境可导入 torch 和 torchvision。系统依据同帧原图及检测框计算并显示深度图与估计距离；板端结果显示后，仍需等待电脑后台分析结束，再回放或导出。

步骤 12　验证摄像头输入：确认照片组任务已结束、板卡 READY，选择设备编号并点击“打开摄像头”，待左侧出现有效预览后选择模式，再点击“开始摄像头运行”。此处不使用照片组的“开始运行”。用船舶图像或现场画面检查结果随输入变化；“暂停/继续”控制摄像头会话，“重置追踪”重新分配 ID。结束时点击“停止摄像头运行”或“关闭摄像头”，待板卡就绪后再切换任务。

步骤 13　回放并保存复现结果：任务和后台分析结束后，使用上一帧、下一帧、滑块或播放控制检查结果，点击警示记录核对触发帧，再执行“导出视频”。新生成的单组结果位于 src/vitis/runs/<运行编号>/frames.jsonl，合集结果位于 src/vitis/runs/COLLECTION_<编号>/frames.jsonl；后续使用“打开结果”载入，并保留匹配原图。使用本次运行生成的数据进行回放。记录所用镜像、输入清单、模式、阈值、阶段耗时与异常日志，作为本次复现材料。

步骤 14　核对启动镜像组成：需要重新封装现有启动组件时，在 Vivado/Vitis 2019.2 工具环境中执行 build/package_boot.cmd。board/boot.bif 按顺序引用 components/fsbl_ethernet_2019_2.elf 、 components/design_1_wrapper.bit 和

components/haiyun_eth_app.elf，生成 board/BOOT_rebuilt.bin。核对 BIF 引用的三个组件后再打包；测试生成的镜像前备份原 BOOT.bin。

4.4 大模型协作记录

本项目使用大模型辅助开展代码阅读、软硬件接口核查、算法调试、验证脚本编写与工程资料整理。协作过程中，团队负责提出应用需求、确定功能取舍、执行板卡操作和人工标注；模型负责将问题分解为可检查的技术环节，通过读取源码、运行工具及比较输出提出修改建议。方案是否采纳，以代码差异、测试日志、综合报告和实板结果为依据，模型生成的解释不单独作为功能正确或性能提升的证明。

（1）协作方式与典型任务

项目采用“提出问题—检查证据—形成方案—独立验证—保留或否决”的工作流程。对于已经跑通的系统，先保留源码、参数和启动文件，再在独立副本中修改；对于算法比较，固定检测输入和人工标注，避免同时改变多个条件而无法解释结果。

典型协作工作按技术问题、实施方法和验证依据归纳于表 4-2，具体纠错过程与方案取舍在后文展开。

表 4-2　典型技术问题与协作验证

技术问题	AI 分析与实施	采纳依据与验证
轨迹状态与显示框耦合，导致框位置滞后或尺寸收缩	沿检测、关联、绘制链路检查；利用 detection_index 关联当前检测框，保留卡尔曼状态用于后续匹配。	采纳状态与显示分离方案；分别评估内部框和显示框，不将绘制改善等同于检测精度提升。
遮挡与交会条件下的身份关联稳定性	冻结检测输入，独立编译跟踪器；按场景重置状态，逐帧核对 ID、匹配索引及框坐标。	17 组 189 帧重放与原输出一致；保留 G111 两次 ID 切换作为回归问题，未将主机验证等同于实板测试。
模型参数转换与 HLS 算子边界行为的可复现性	复核量化与 BN 融合；建立独立整数参考和输出哨兵检查，记录边界修复及同配置综合结果。	42 个参数文件字节一致；修复后 69,121 个卷积输出及哨兵检查通过。共享出口未降低资源，未采纳其优化假设。

（2）采纳的建议及其工程落实

跟踪框偏窄、位置滞后最初表现为界面问题，但其原因需要从检测、关联和绘制三个环节分别检查。协作中，模型提出：ByteTrack 内部框用于运动预测与关联，显示框则可使用本帧匹配到的检测框。该方案通过 detection_index 建立轨迹与检测之间的对应关系，保持目标 ID，同时减少内部滤波状态滞后对显示的影响。

项目采纳了这一分工，但没有将其解释为检测精度提高。若检测器本身漏检或只输出局部船体，替换绘制框并不能恢复完整目标。因此，后续评估分别统计内部框与显示框的结果，避免把显示改善误认为关联算法全面改善。

模型与硬件接口核查也采用了相同思路。针对 SD 浮点输入与以太网整数输入是否等价的问题，独立验证穷举了 256 种像素取值，并对 3 张实际图像进行预处理对照；针对参数导出，重新执行 19 个层的 BN 融合核验，导出的 42 个参数文件均与历史文件字节一致。团队采纳了“先验证格式与数值约定，再判断算法差异”的方法。由于这些检查没有发现转换错误，保留了原实现，没有为了形成修改记录而改动已经正确的代码。

（3）自我纠错与验证过程

HLS 复现提供了较为完整的纠错案例。初期数值测试通过后，协作流程继续增加输出缓冲区哨兵检查，发现单 tile 的 3×3 卷积虽然有效输出数值正确，却改写了有效区域之后的 5408 个元素。进一步检查循环控制发现，程序完成首个 tile 后仍无条件进入下一轮，退出判断发生过晚。

模型在独立副本中修正了循环边界，随后 8 项测试共 69,121 个有效输出逐位一致，输出哨兵检查也通过。这说明验证不能只比较“应当得到的结果”，还需要检查“不应发生的写入”。

该过程中也出现了模型新增测试代码自身的问题：局部变量名与原头文件宏定义冲突，导致编译失败。记录中保留了失败原因、修改方式和重跑结果，避免将测试工具错误误记为产品缺陷。

对于修复方案，模型还提出共享存储出口可能减少硬件重复逻辑的假设。但同配置 HLS 综合结果显示，共享出口版本并未比提前返回版本减少资源，LUT 估算反而由 103092 增加至 103141。因此，团队未采纳“该调整带来资源优化”的结论，只将其作为控制流程表达与边界修正进行记录。该案例属于独立算子实验，不能直接解释为现有船舶漏检或串 ID 问题已经得到修复。

（4）未采纳或暂缓采纳的建议

团队对模型建议的筛选不仅考虑功能可行性，也考虑验证成本、板端开销和结论适用范围。主要取舍如表 4-3 所示。

表 4-3　模型建议的取舍与依据

建议或初步判断	最终处理	判断依据
通过局部复检改善重叠混框和漏检	未作为本次主线的必需步骤	额外推理增加计算开销，需要独立验证触发条件及收益；保留为扩展实验。
共享存储出口能够降低 HLS 资源占用	未采纳该性能结论	同条件综合数据未支持资源下降。
依据 120 MHz 综合约束描述发布系统运行频率	修正表述	发布 FSBL 初始化数组核查指向 100 MHz 配置；约束目标不能代替发布配置。
由相对深度直接得到船间距离或碰撞风险	未采纳直接报警方案	缺少可靠尺度、相机标定及现场距离真值，深度功能保留为估计扩展。
主机重放通过即可说明新版本实板通过	未采纳	主机执行、ARM 构建、BOOT 封装及实板运行属于不同验证阶段。

其中，时钟核查没有停留在工程目录名称或旧报告，而是进一步比较发布 ELF 中的 PS 初始化数组。15 个初始化数组与对应源文件匹配，支持发布固件采用 FCLK0 为 100 MHz 的配置结论。

（5）验证结果与团队经验

团队将这些经历整理为以下工程认识。

① 先明确比较条件，再讨论优化收益。输入尺寸、数据布局、量化尺度、检测阈值和运行时钟都可能改变结果，必须区分算法变化与测试条件变化。

② 把正确性分层验证。编译成功、数值一致、内存访问安全、时序通过和实板运行成功分别回答不同问题，不能相互替代。

③ 保留不支持预期的结果。某项修改没有减少资源，或某个场景仍有 ID 切换，也是确定方案边界的重要依据。

④ 把经验固化为开发流程和辅助工具。最新版 Skill 将输入接口、构建依赖、失败定位、修改记录和测试结果组织在同一目录中，并提供文件比较、数值对照及实现报告处理脚本，使后续成员能够按实际工程条件继续开发。

⑤ 人的核心作用在于确定目标与验收边界。模型可以扩大代码检查和方案探索的范围，团队仍需依据实际使用需求决定哪些功能进入主线、哪些保留为实验，以及哪些证据足以支持报告结论。

第五部分　参考文献

[1] BOCHKOVSKIY A, WANG C Y, LIAO H Y M. YOLOv4: Optimal Speed and Accuracy of Object Detection[EB/OL]. 2020[2026-11-04]. https://arxiv.org/abs/2004.10934

[2] ZHANG Y, SUN P, JIANG Y, et al. ByteTrack: Multi-Object Tracking by Associating Every Detection Box[C]//European Conference on Computer Vision. 2022: 1-21. https://arxiv.org/abs/2110.06864

[3] YANG L, KANG B, HUANG Z, et al. Depth Anything V2[C]//Advances in Neural Information Processing Systems. 2024. https://arxiv.org/abs/2406.09414

[4] ULTRALYTICS. YOLOv5: v6.0[CP/OL]. 2021[2026-11-04]. https://github.com/ultralytics/yolov5/tree/v6.0

[5] SHAO Z, WU W, WANG Z, et al. SeaShips: A Large-Scale Precisely Annotated Dataset for Ship Detection[J]. IEEE Transactions on Multimedia, 2018, 20(10): 2593-2604. DOI: 10.1109/TMM.2018.2865686. https://sites.ucmerced.edu/files/wdu/files/seaship.pdf

[6] YANG L, ZHANG R Y, LI L, XIE X. SimAM: A Simple, Parameter-Free Attention Module for Convolutional Neural Networks[C]//Proceedings of the 38th International Conference on Machine Learning. 2021, 139: 11863-11874. https://proceedings.mlr.press/v139/yang21o.html

[7] WANG J, CHEN K, XU R, et al. CARAFE: Content-Aware ReAssembly of FEatures[C]//IEEE/CVF International Conference on Computer Vision. 2019: 3007-3016. https://arxiv.org/abs/1905.02188

[8] ZHU X, HU H, LIN S, DAI J. Deformable ConvNets v2: More Deformable, Better Results[C]//IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2019: 9308-9316. https://arxiv.org/abs/1811.11168

[9] TUL CORPORATION. PYNQ-Z2 Schematics: TUL_PYNQ_Schematic_R12[EB/OL]. 2018[2026-11-04]. https://dpoauwgwqsy2x.cloudfront.net/Download/TUL_PYNQ_Schematic_R12.pdf

[10] AMD. Zynq 7000 SoC Technical Reference Manual: UG585, v1.15[EB/OL]. 2026-02-06[2026-11-04]. https://docs.amd.com/r/en-US/ug585-zynq-7000-SoC-TRM

[11] XILINX. Vivado Design Suite User Guide: High-Level Synthesis: UG902[EB/OL]. 2019-07-12[2026-11-04]. https://docs.amd.com/v/u/2019.1-English/ug902-vivado-high-level-synthesis

[12] JEONG M, CHADDA A, REN Z, et al. SeePerSea: Multi-modal Perception Dataset of In-water Objects for Autonomous Surface Vehicles[J]. IEEE Transactions on Field Robotics, 2025, 2: 737-752. DOI: 10.1109/TFR.2025.3602937. https://arxiv.org/abs/2404.18411